# Introduction to Operating System




By:

**Dr. H. K. Azad**
Assistant Professor (Sr.)
SCOPE
Email: hiteshwarkumar.azad@vit.ac.in

# Content

- Course Introduction
  - Syllabus
  - Course Structure
  - Textbook(s)
  - Reference books
  - Web References/MOOCs
- What is an Operating System?
  - Bootstrap process
  - Abstract view
  - Why do we need an operating system?
  - Computer System Components
  - Types of OS and its features

- Textbook
  - Abraham Silberschatz, Peter B. Galvin, Greg Gagne-Operating System Concepts, Wiley (2012).
- Reference books
  - Ramez Elmasri, A Carrick, David Levine, Operating Systems, A Spiral Approach - McGrawHill Science Engineering Math (2009).
  - Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau, Operating Systems, Three Easy Pieces,Arpaci-Dusseau Books, Inc (2015).
- Web References/MOOCs
  - https://nptel.ac.in/courses/106/105/106105214/
  - https://www.tutorialspoint.com/operating_system/index.htm
  - https://www.javatpoint.com/os-tutorial

# Assessment Methods

| Assessment type | Max. Marks | Weightage |
|---|---|---|
| *Assignment 1 | 10 | 10 |
| Quiz 1 | 10 | 10 |
| Quiz 2 | 10 | 10 |
| CAT – I | 50 | 15 |
| CAT – II | 50 | 15 |
| FAT | 100 | 40 |
| | Total | 100 |

*Assignment 1 → Case Study [Due date : 8th October 2026]

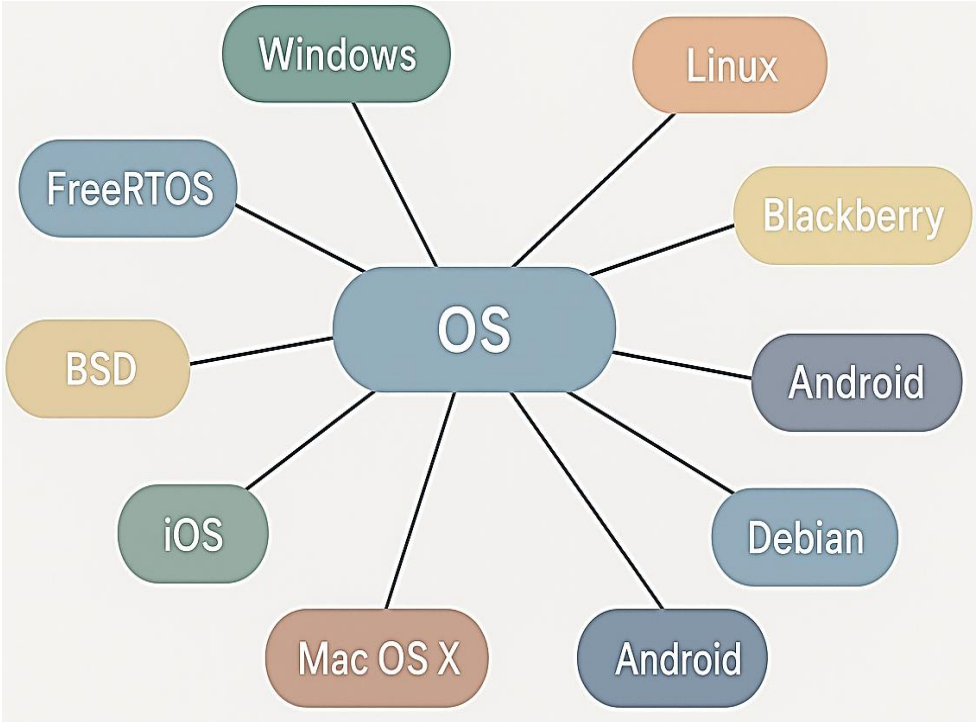
# Model 1: Introduction

- What is an Operating System?
- Mainframe Systems
- Desktop Systems
- Multiprocessor Systems
- Distributed Systems
- Clustered System
- Real -Time Systems
- Handheld Systems
- Feature Migration

# What is an Operating System?



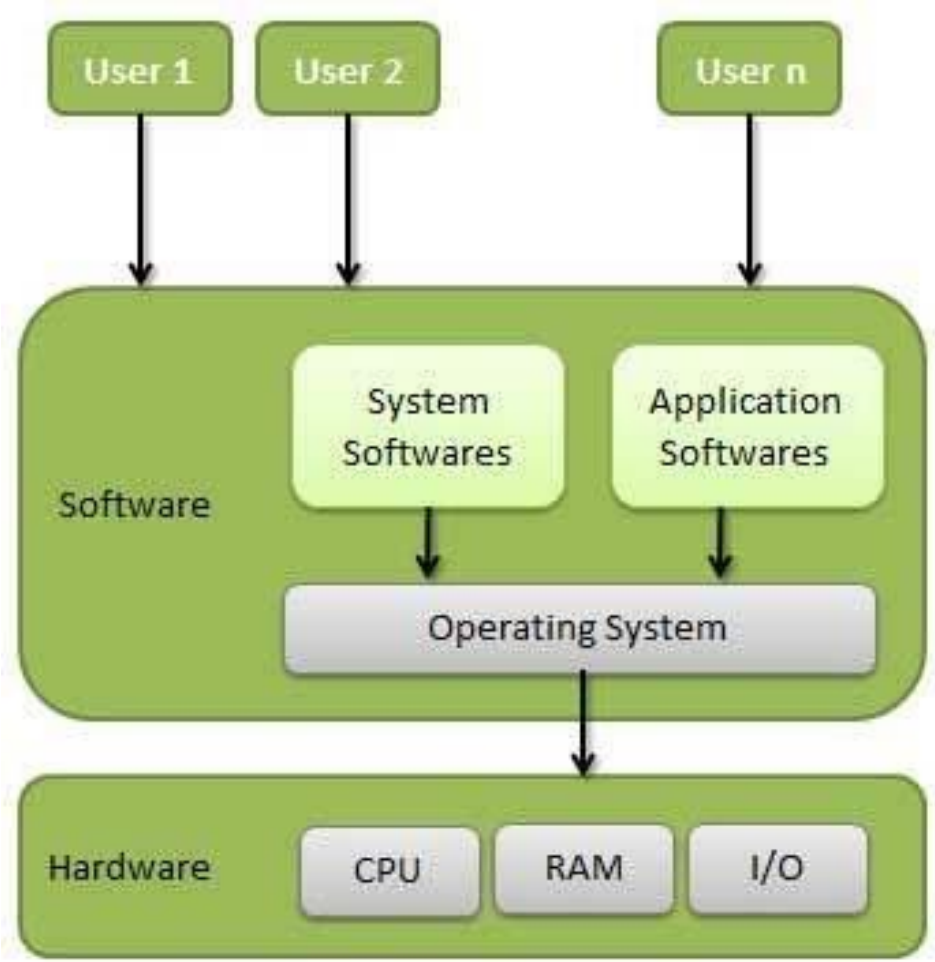

- An Operating System (OS) is a program that manages the computer hardware.
- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - Execute user programs and make solving user problems easier
  - Make the computer system convenient to use
  - Use the computer hardware in an efficient manner

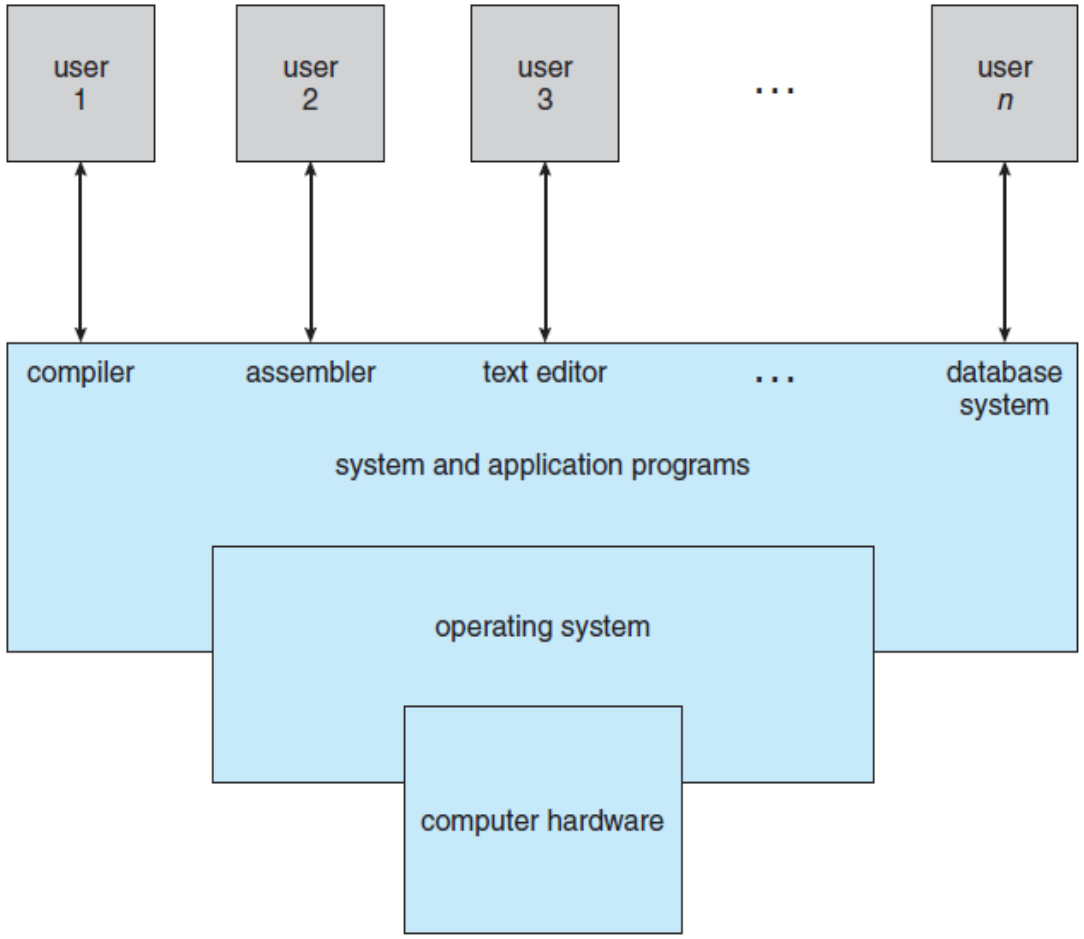


# Computer System Components

- Computer system can be divided into four components:
  - Hardware – provides basic computing resources
    - CPU, memory, I/O devices
  - Operating system
    - Controls and coordinates use of hardware among various applications and users
  - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, compilers, web browsers, database systems, video games
  - Users
    - People, machines, other computers

# Abstract View of System Components



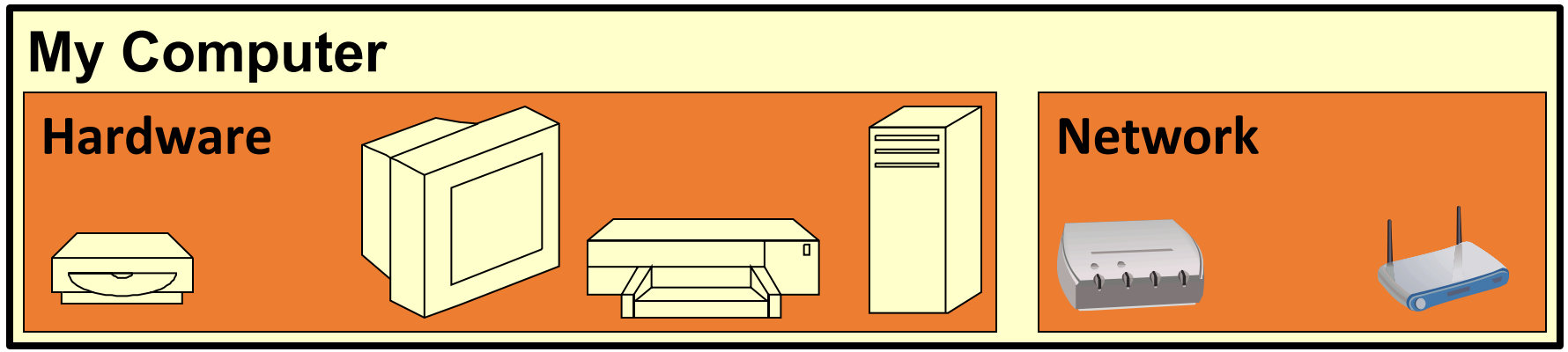



# Goals of an Operating System

- Simplify the execution of user programs and make solving user problems easier.
- Use computer hardware efficiently.
- Allow sharing of hardware and software resources.
- Make application software portable and versatile.
- Provide security and protection among user programs.
- Improve overall system reliability.

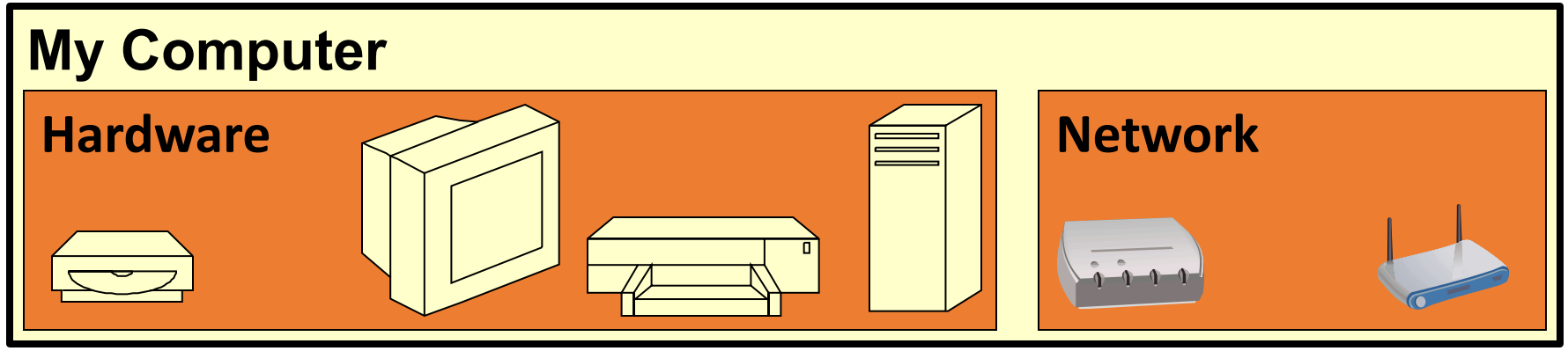
# Why do we need an operating system?

- What do we have?
  - Set of common resources

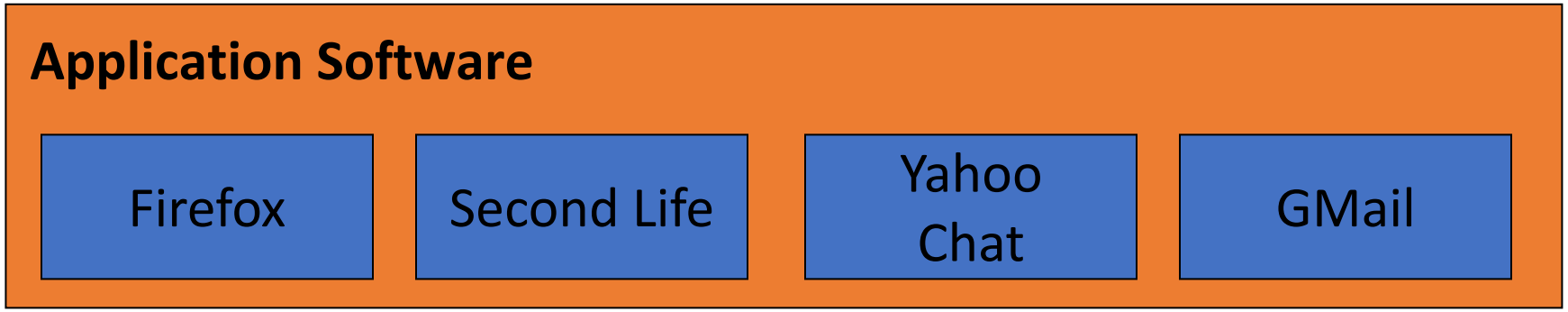


# Why do we need an operating system?

- What do we have?
  - Set of common resources
- What do we need?



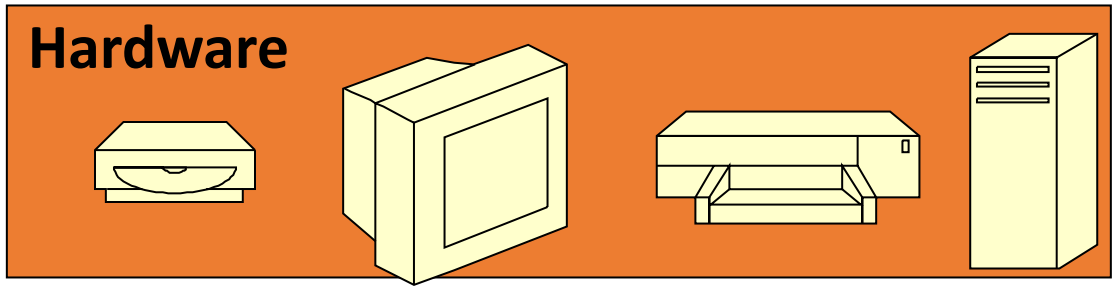
# Why do we need an operating system?

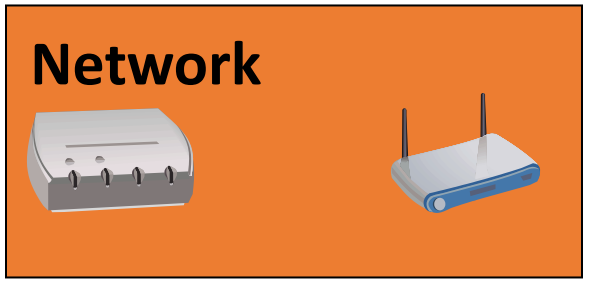



What do we have?

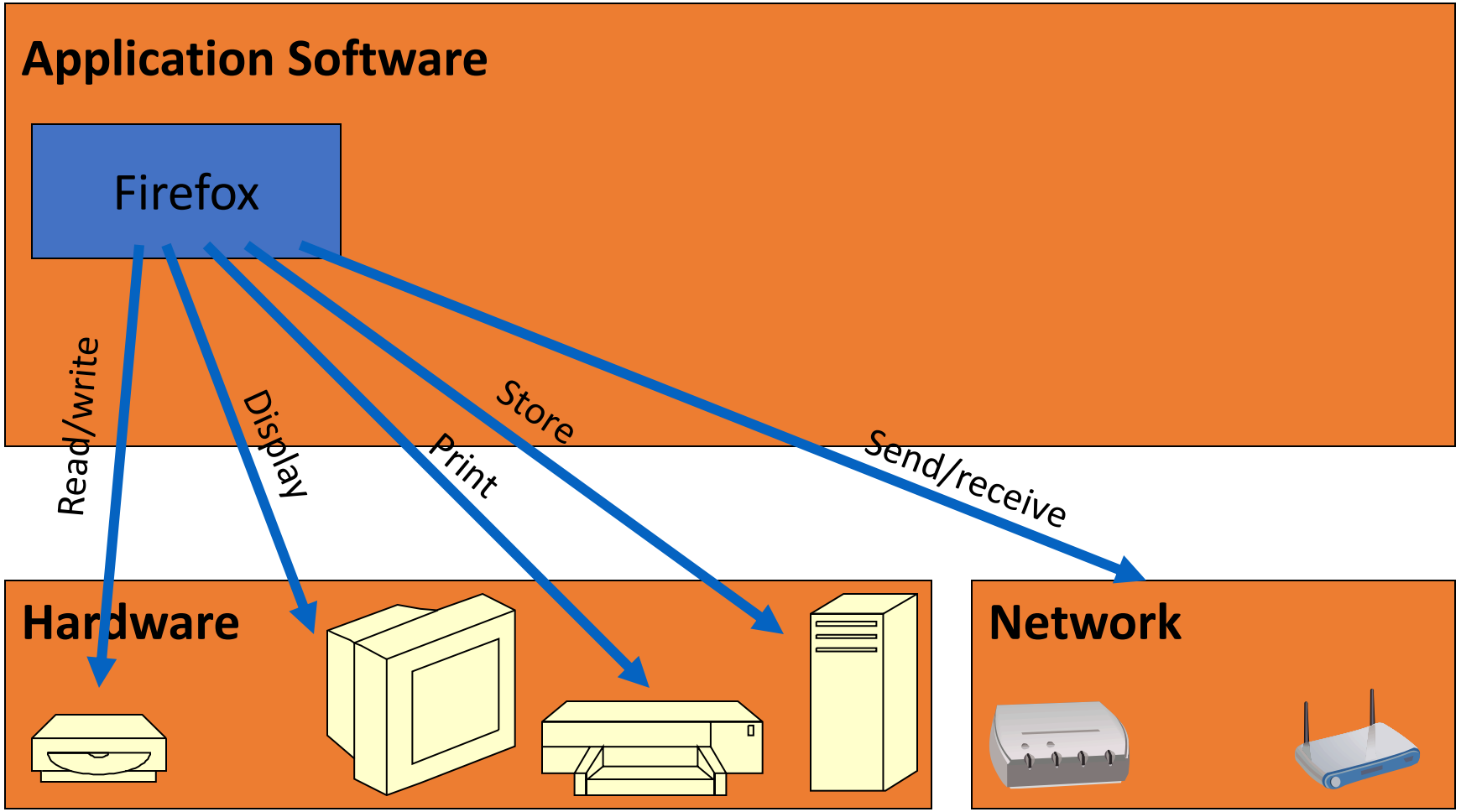
- Set of common resources

- What do we need?
  - A clean way to allow applications to use these resources!

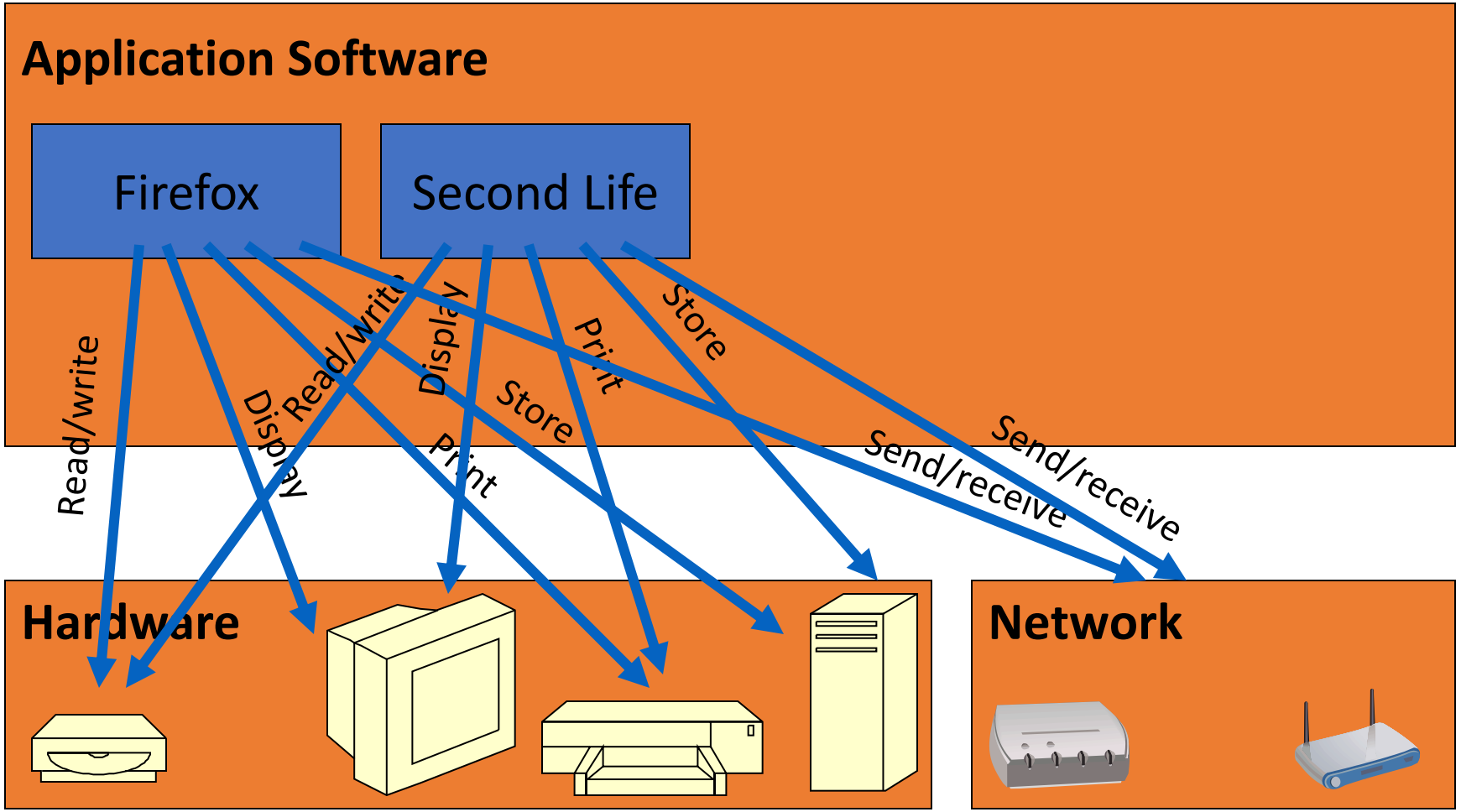


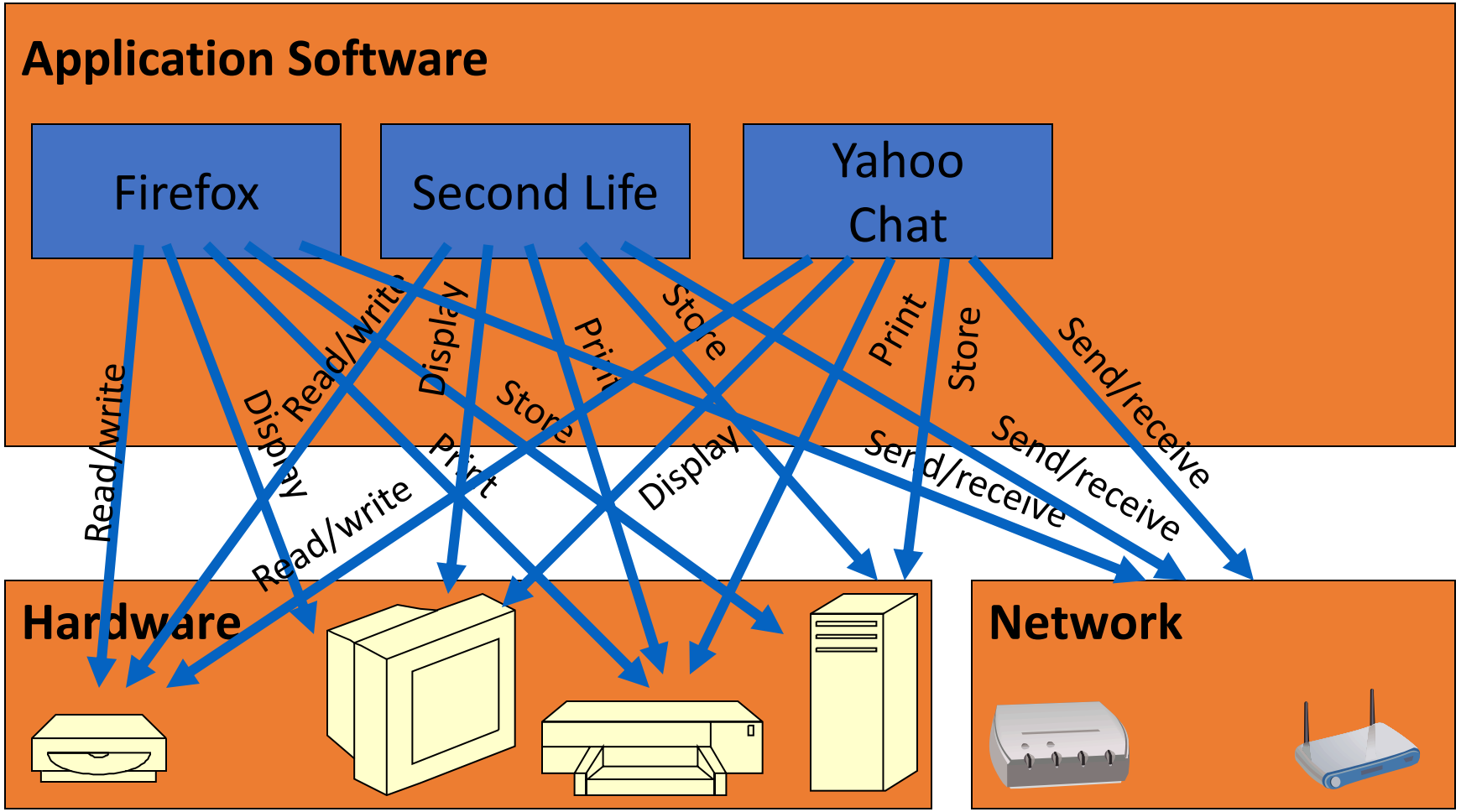
# Application Requirements



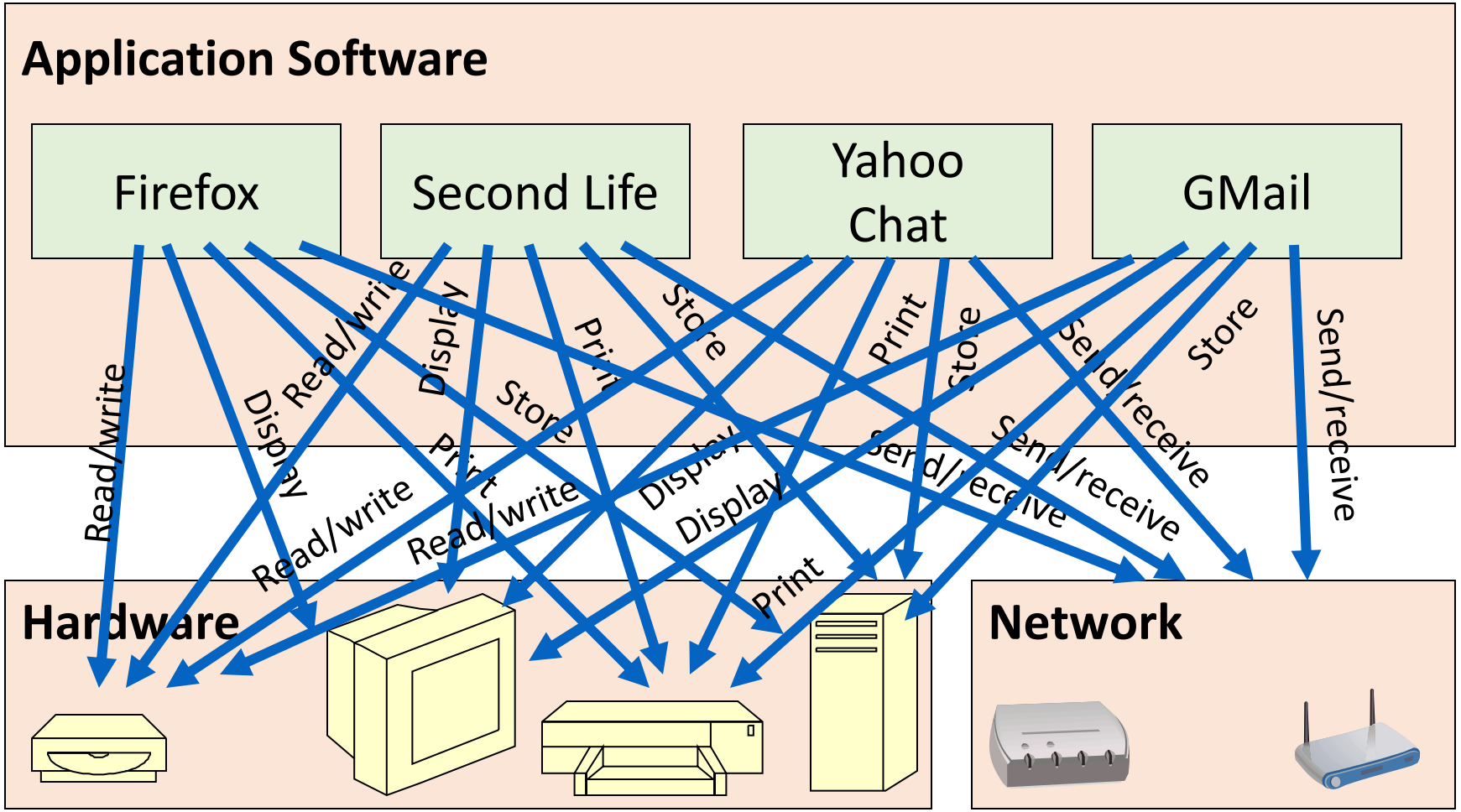
# Two Applications?



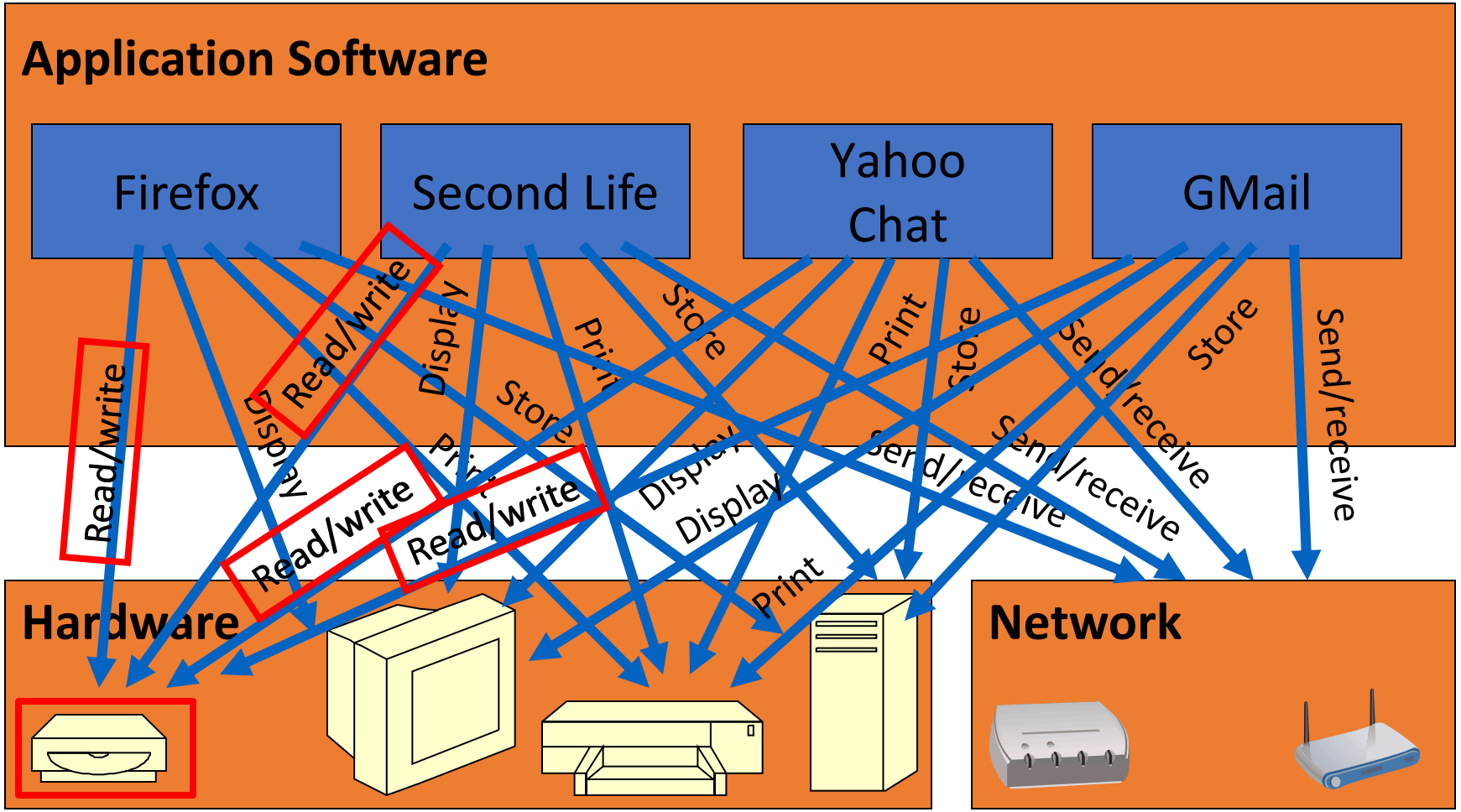
# Managing More Applications?



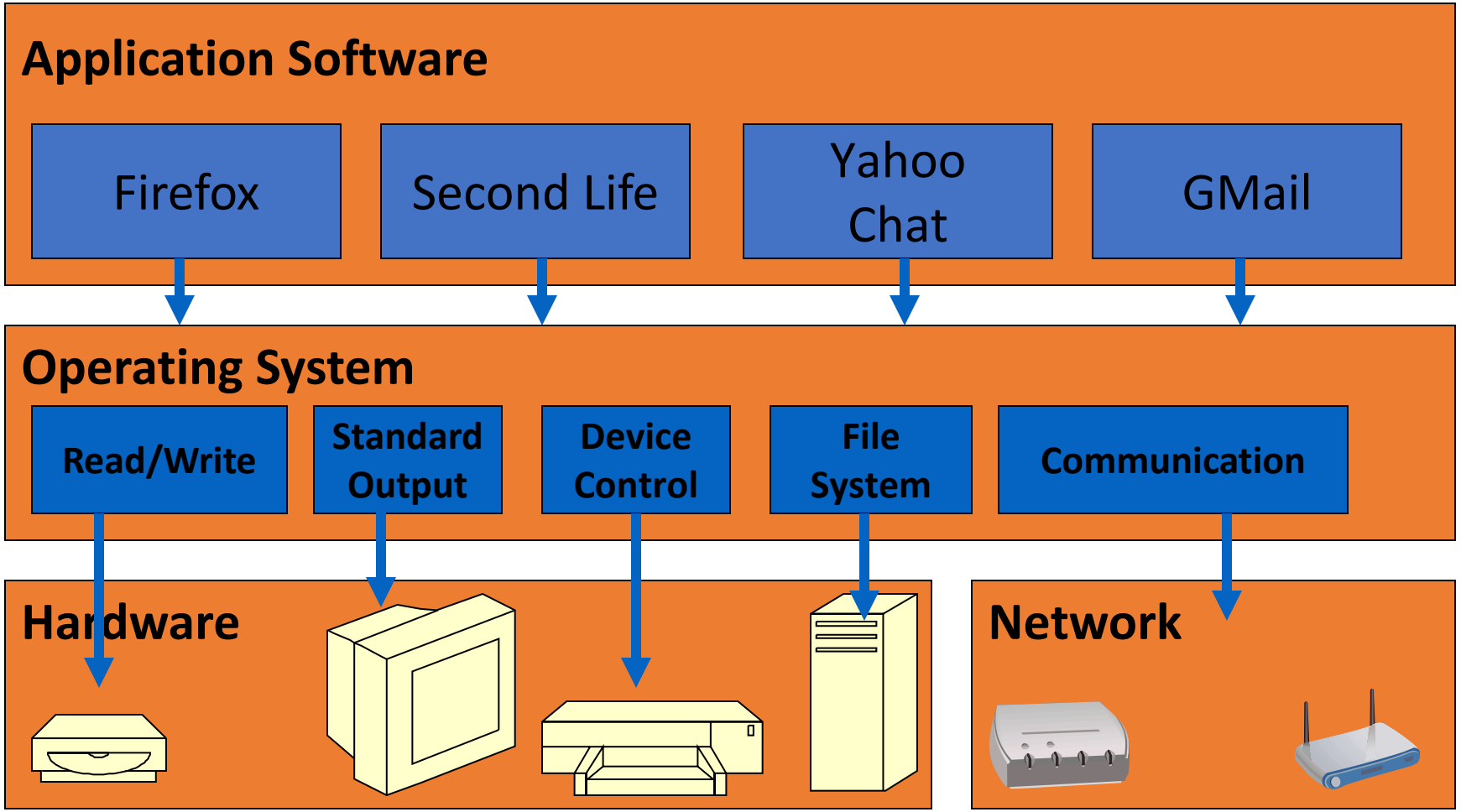
# *We need help!*


Application Software
Firefox
Second Life
Yahoo Chat
GMail
Read/write
Display
Store
Print
Send/receive
Hardware
Network

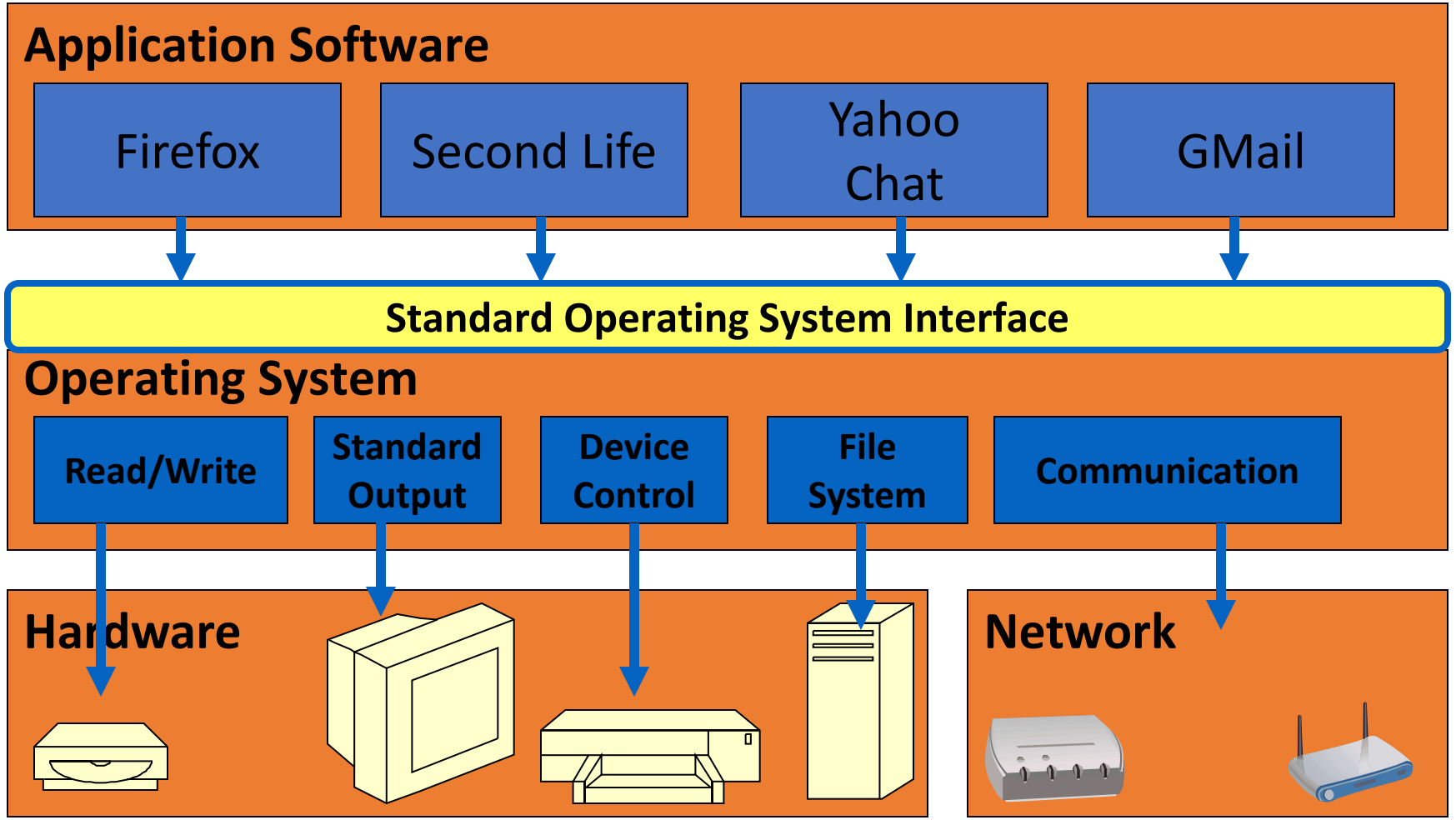
# Approach: Find Common Functions



# Delegate Common Functions



# Export a Standard Interface



# Operating System: Key Definitions



## OS as a Resource Allocator

- Manages and distributes hardware and software resources (CPU time, memory, I/O devices, storage, etc.)
- Resolves competing requests to ensure **efficient** and **fair** resource usage



## OS as a Control Program

- Oversees the execution of programs
- Enforces rules to prevent errors, conflicts, and improper use of the computer system

# Operating System Definition (Cont.)

 **No universally agreed-upon definition of an operating system**

- Different vendors and authors include different components under “OS.”

 **A practical working definition:**

*“The collection of software delivered by a vendor when you install an operating system.”*

- This definition is useful, but the actual contents **vary widely** across systems.

 **The Kernel:**

*“The fundamental program that is always running on the computer.”*

- Manages hardware interactions
- Provides core services such as process scheduling, memory management, and device control

 **Everything else belongs to one of two categories:**

- **System programs** — tools and utilities that support or extend OS functionality
- **Application programs** — software that directly serves user needs (e.g., browsers, editors)

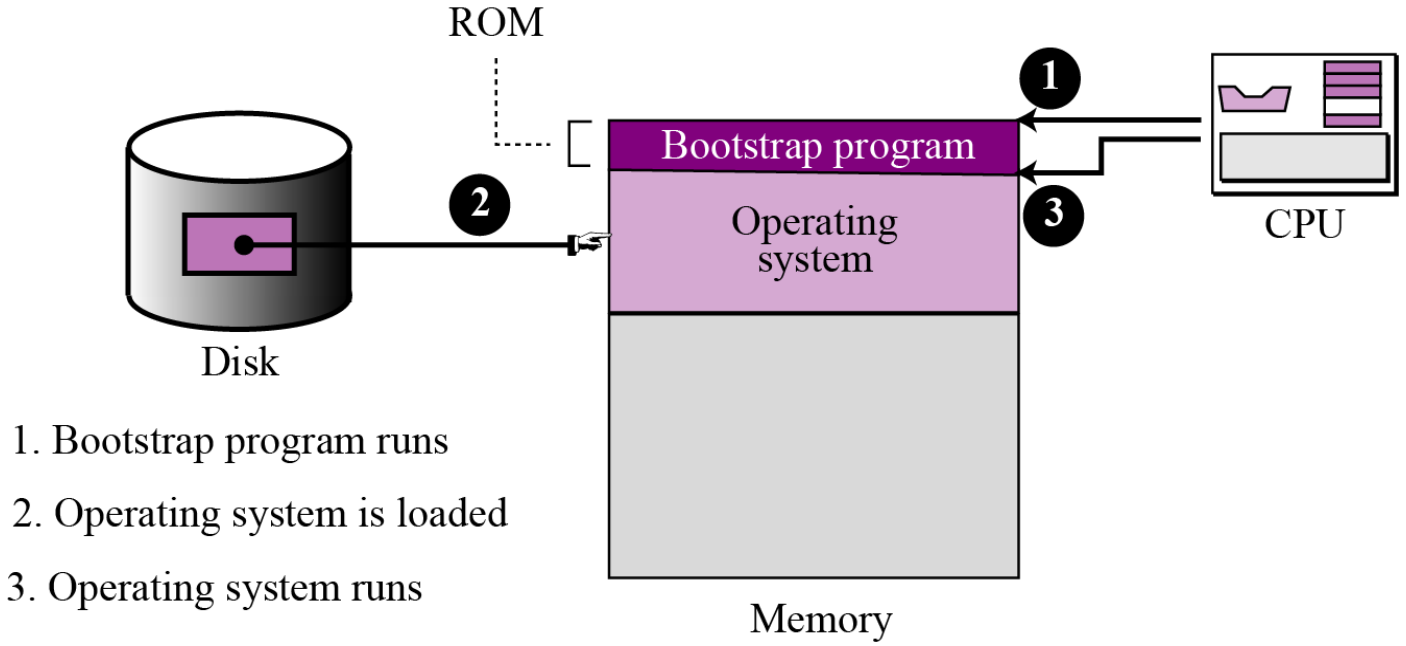
# Computer Startup

**Bootstrap program** is executed when the computer powers on or reboots

- Stored in **ROM**, **EPROM**, or similar non-volatile memory — collectively known as **firmware**
- Performs initial hardware checks and system setup
- Locates the operating system on disk, loads the **kernel** into memory, and transfers control to it

**Startup sequence:**

1. **Bootstrap program runs** from firmware
2. **Operating system kernel is loaded** from disk into memory
3. **Operating system begins executing** and takes control of the system



# Introduction to Linux Shell and Kernel

## What is the Kernel?

- The **kernel** is the **core component** of the operating system.
  It has complete control over the system's hardware and manages all interactions between hardware and software.

## Responsibilities of the Linux Kernel

-  File management
-  Process management
-  I/O management
-  Memory management
-  Device management

## Common Misconceptions

- Many believe **Linus Torvalds created the Linux operating system**.
  – In reality, he developed the **Linux kernel**.
  – The full Linux system also includes GNU utilities and many other tools.

## What Makes a Complete Linux System?

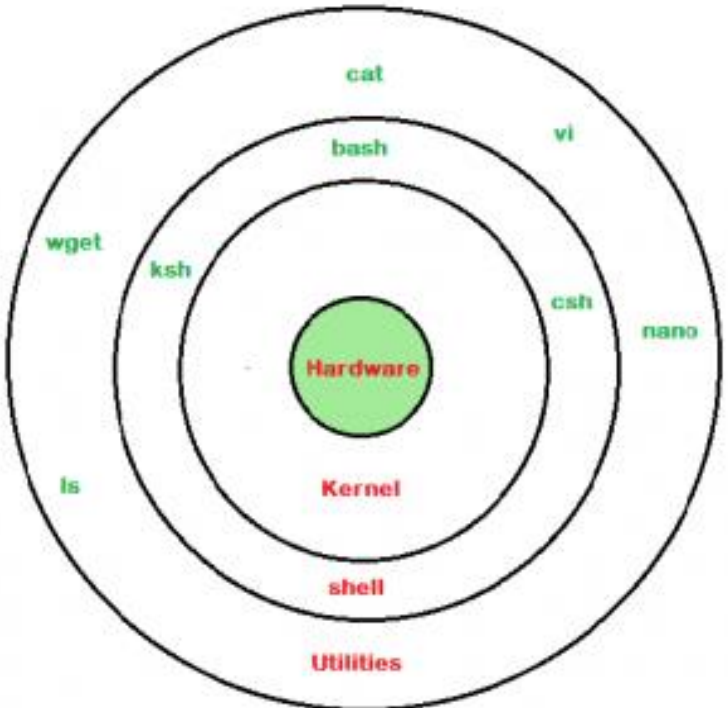
- **Linux system = Linux Kernel + GNU system utilities and libraries + additional tools and scripts**

# What is a Shell?

- 💻 **Shell as Interface** — A shell is a user-level program that provides an interface for users to access operating system services.
- 💻 **Command-Line Interpreter** — In Linux and other Unix-like systems, the shell provides a **Command-Line Interface (CLI)** for interacting with the kernel.
- 🔤 **Translates Commands** — It accepts **human-readable commands** and converts them into low-level instructions understood by the kernel.
- ⚙️ **Command Executor** — Acts as a command language interpreter, executing commands typed by the user or read from scripts.
- 🚀 **Starts Automatically** — The shell launches when a user logs in or opens a terminal session.

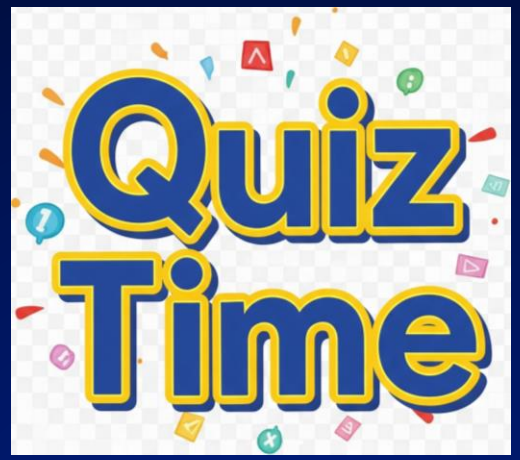
# Types of Shells

- ⌨️ **Command-Line Shell** — e.g., Bash, Zsh, Ksh, Fish
- 🖼️ **Graphical Shell** — e.g., GNOME Shell, KDE Plasma



# Shell vs Kernel — Comparison Table

| Feature / Aspect | **Kernel** 🧠 | **Shell** ⌨️ |
|---|---|---|
| **Definition** | Core part of the OS that manages system resources | Interface that allows users to interact with the OS |
| **Role** | Controls hardware, processes, memory, and devices | Accepts commands from users and sends them to the kernel |
| **Position in OS** | Sits between hardware and shell/user processes | Sits between the user and the kernel |
| **Mode of Operation** | Runs in *kernel mode* (privileged mode) | Runs in *user mode* |
| **Functionality** | Resource management, scheduling, I/O control, security | Command interpretation, script execution, user interaction |
| **Visibility to Users** | Not directly visible to users | Directly used by users through CLI or GUI |
| **Types** | No end-user types (architecture only: monolithic, microkernel, etc.) | Many types: Bash, Zsh, C Shell, Korn Shell, Fish, graphical shells |
| **When It Starts** | Loads at system boot time | Starts when a user logs in or opens a terminal |
| **Dependency** | OS cannot function without the kernel | Shell depends on kernel to execute commands |




START THE QUIZ

What is the main purpose of an operating system?

A) Compiling programs

C) Managing hardware resources

B) Editing text files

D) Connecting to the internet

Next

**Which of the following is not a function of the operating system?**

A) Memory management

C) File management

B) Compiler design

D) Process scheduling

Next

**The part of the OS that manages CPU execution and process switching is:**

A) Shell

C) BIOS

B) File system

D) Kernel

Next

**Which of the following allows multiple users to run programs simultaneously?**

A) Single-tasking system

C) Multi-user operating system

B) Real-time system

D) Batch system

Well done!

# Operating System Structures

- Operating systems are designed using different structural models that organize system components and determine how they interact.
- The four primary OS structures:
  - Monolithic
  - Layered
  - Microkernel
  - Modular

# Monolithic OS Structure

- A single large kernel contains all core OS services (process management, memory management, file system, device drivers).

**Key Characteristics:**

- All components share one address space (kernel mode)
- Tight integration of system services
- Direct system call handling

**Advantages:**

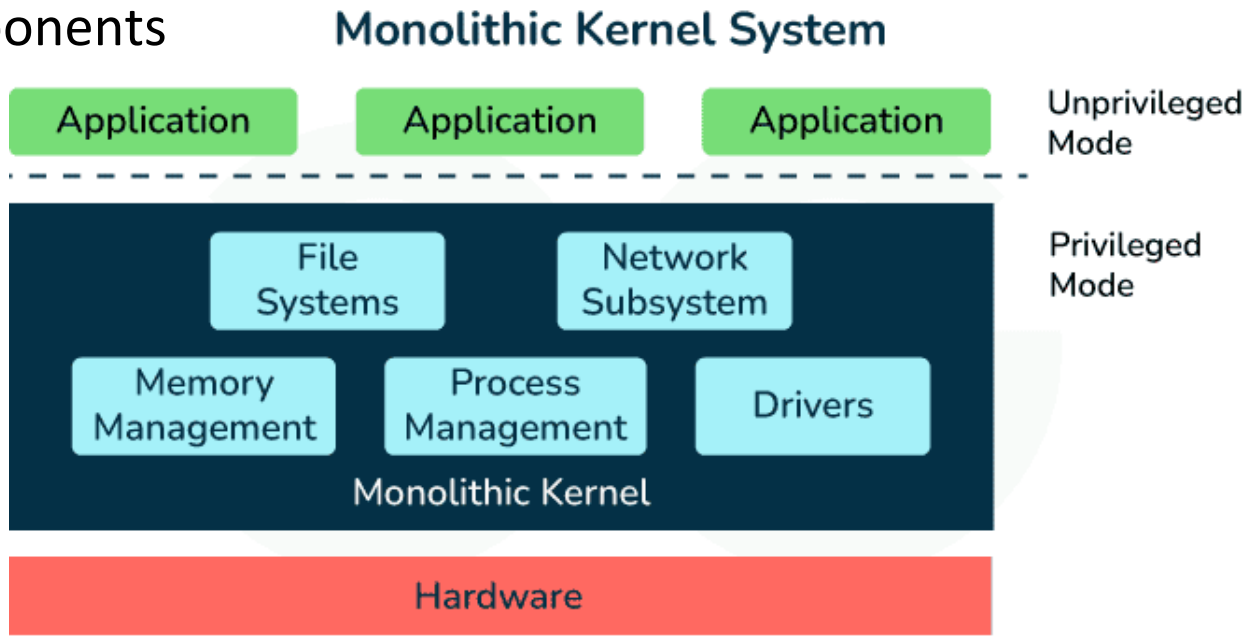
- High performance and fast execution
- Low communication overhead between components

**Disadvantages:**

- Hard to maintain and debug
- Single fault can crash the entire system
- Poor modularity

**Examples:**

- Traditional UNIX
- Early Linux versions



# Layered OS Structure

**Definition:**

- OS is organized into multiple layers, each built on top of the one below.

**Key Characteristics:**

- Clear separation of responsibilities
- Each layer uses services from lower layers
- Top layer provides user interface; bottom interacts with hardware

**Advantages:**

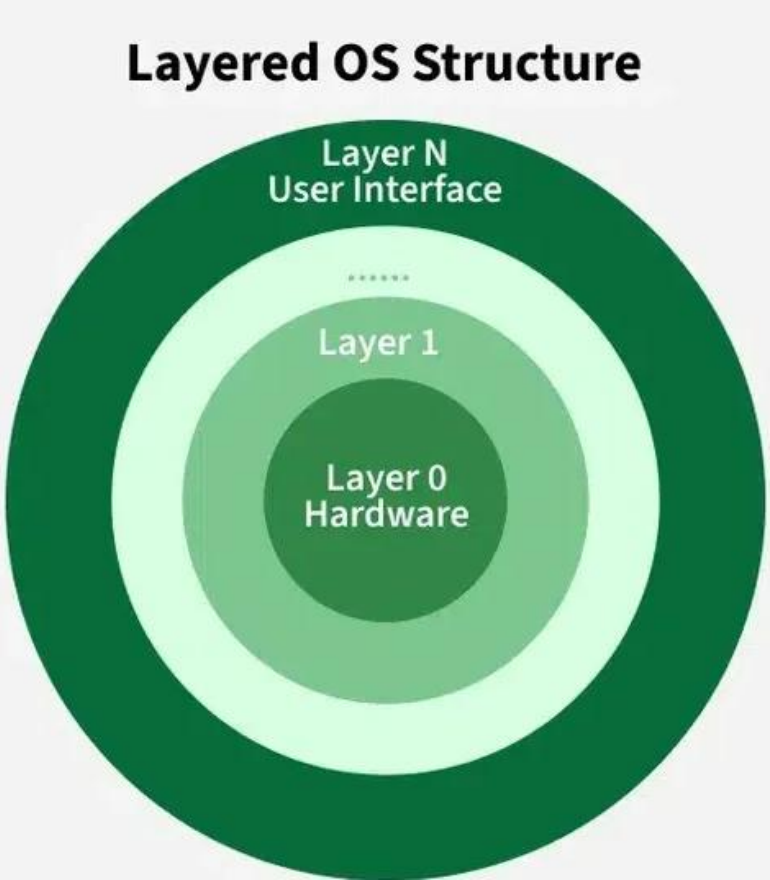
- Easy to design, test, and debug
- Well-defined interfaces and abstractions

**Disadvantages:**

- Can introduce performance overhead
- Difficult to design strict, logical layer boundaries

**Examples:**

- THE OS
- Windows NT (partially layered)



# Microkernel OS Structure

**Definition:**

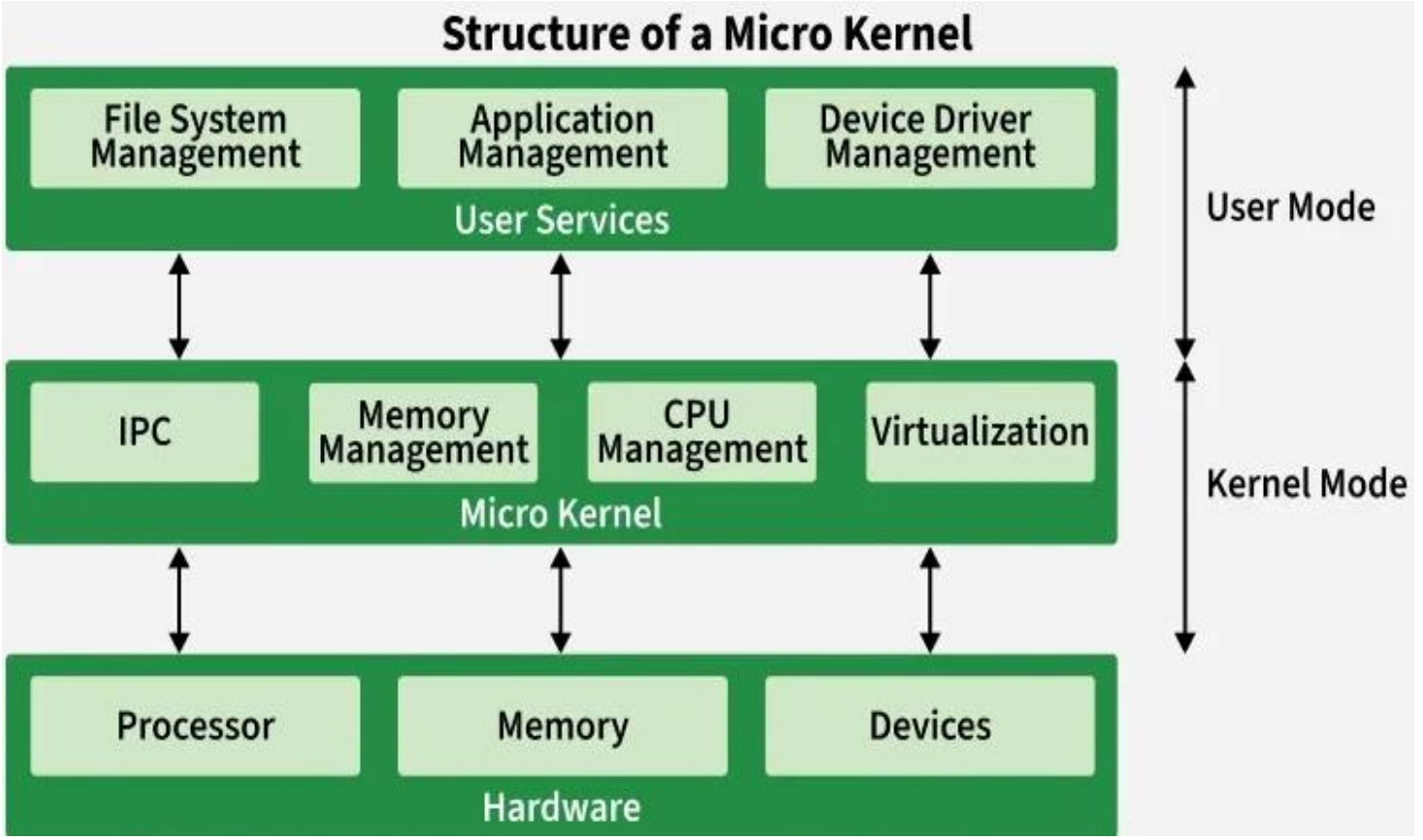
- Minimal kernel containing only essential services; other services run in user space.

**Kernel Contains:**

- Basic scheduling
- Memory management
- Inter-process communication (IPC)

**User-Space Services:**

- Device drivers
- File systems
- Network services




**Advantages:**

- High reliability and security
- Easier to extend and maintain
- Fault isolation improves system stability

**Disadvantages:**

- Higher overhead due to IPC
- More complex communication between components

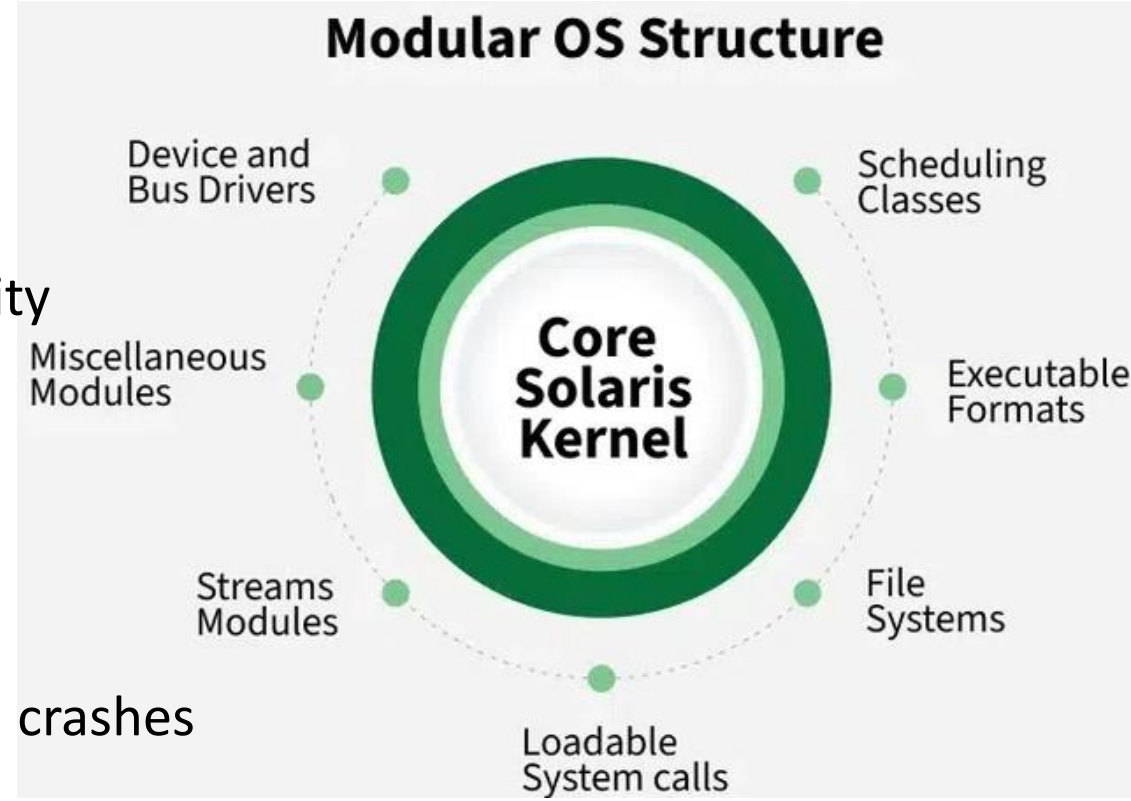
**Examples:**

- Minix 3, QNX, macOS (hybrid microkernel)

# Modular OS Structure

**Definition:**

- OS kernel supports dynamically loadable modules that extend functionality.




**Key Characteristics:**

- Core kernel + loadable modules
- Modules can be added/removed at runtime
- Combines monolithic speed with microkernel flexibility

**Advantages:**

- Easy maintenance and updates
- Scalable and extensible
- Good performance

**Disadvantages:**

- Modules run in kernel mode → can still cause system crashes
- Requires careful design of module interfaces

**Examples:**

- Modern Linux
- Solaris

# Comparison Table – OS Structures

| Feature / Structure | **Monolithic** | **Layered** | **Microkernel** | **Modular** |
|---|---|---|---|---|
| **Architecture** | Single large kernel; all services in kernel mode | OS divided into hierarchical layers | Minimal kernel; services in user space | Core kernel + loadable modules |
| **Performance** | Very high | Moderate (layer overhead) | Lower (IPC overhead) | High |
| **Reliability** | Low (one bug crashes system) | Moderate | High (fault isolation) | High |
| **Flexibility** | Low | Moderate | High | High |
| **Maintenance** | Difficult | Easier than monolithic | Easy (components isolated) | Easy (modules replaceable) |
| **Fault Isolation** | Weak | Moderate | Strong | Strong |
| **Examples** | UNIX, early Linux | THE OS, Windows NT (partially) | Minix, QNX, macOS (hybrid) | Modern Linux, Solaris |

# Mainframe Systems

- **Mainframe OSes are** robust, high-performance, multitasking, and multi-user platforms designed to support **massive numbers of transactions, concurrent users, and devices**.

## Key Characteristics

- Designed for large-scale commercial and scientific workloads
- High throughput: handle thousands to millions of operations simultaneously
- Reduce processing overhead by **batching similar jobs**

## Automatic Job Sequencing

- OS automatically transfers control from one job to the next
- Eliminates manual intervention
- Considered the **first form of a basic operating system**

## Resident Monitor (Early OS Concept)

- Monitor begins with initial control
- Control is transferred to the running job
- When the job completes, control returns to the monitor

# Mainframe Systems (cont.)

**Evolution of Mainframe Operating Systems**

- **Simple Batch Systems**
  - Jobs grouped and executed sequentially
  - Minimal user interaction; improved throughput
- **Multiprogrammed Systems**
  - Multiple jobs kept in memory simultaneously
  - CPU utilized more efficiently by switching between jobs
- **Time-Sharing Systems**
  - Many users interact with the system concurrently
  - Rapid switching creates the illusion of dedicated access

# Batch system

- Users **did not interact directly** with the computer system.
- Users prepared a **job** (program + data + control information) and submitted it to an operator.
- The OS automatically **sequenced jobs**, transferring control from one job to the next.
- To improve throughput, operators **batched similar jobs** and executed them together.
- CPU often remained **idle** while waiting for I/O operations.

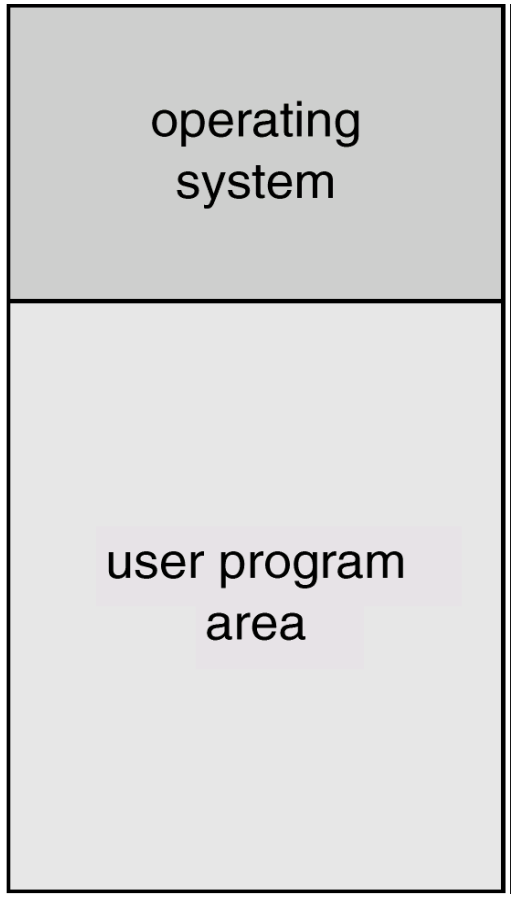
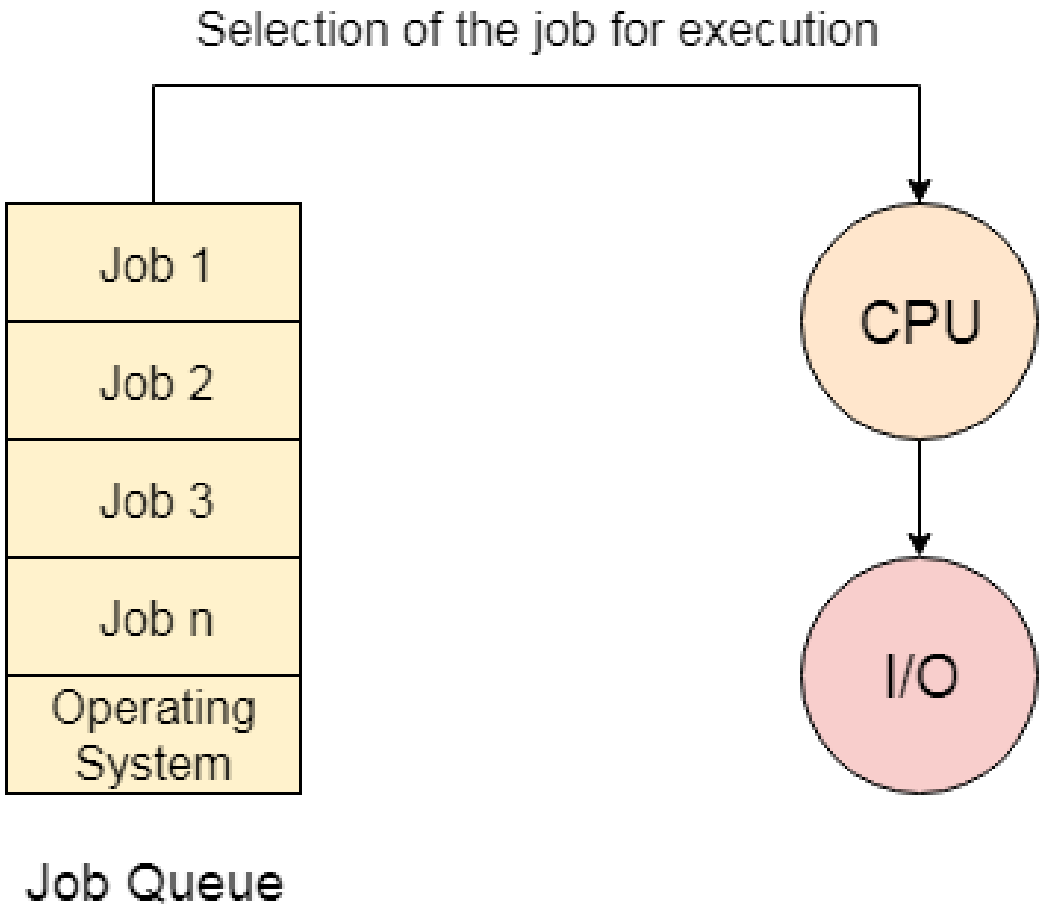



Figure: Memory Layout for a Simple Batch System

# Batch system(cont.)

**First Come, First Serve (FCFS)**

- All submitted jobs are placed in a **queue** in the order they arrive.
- The system executes jobs **sequentially**, one after another.
- No job can start until the previous one finishes.
- Users receive their output **only after all earlier jobs have completed**



# Multiprogrammed System

- Multiple programs are kept in **main memory** simultaneously.
- CPU switches among programs to keep itself **continuously busy**.
- Goal: **maximize CPU utilization**.

## How It Works

- Several jobs reside in memory at the same time.
- The OS selects one job and begins executing it.
- If a job must wait for an I/O operation, the CPU is switched to another job.
- This reduces idle time and improves efficiency.

## Benefits of Multiprogramming

- **Increased CPU Utilization:**
  CPU is never idle while a job waits for I/O.
- **Improved Throughput:**
  More jobs completed in a given time.
- **Better Resource Utilization:**
  Both CPU and I/O devices remain active.

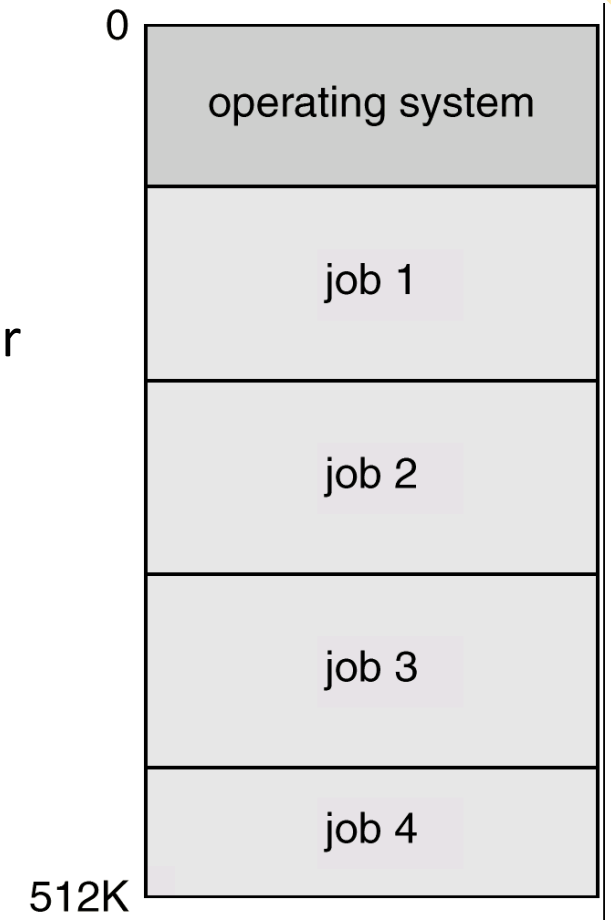



Figure: Memory Layout for a Multiprogramming System

# OS Features Needed for Multiprogramming

- **I/O Management**
  OS must provide routines to handle input/output efficiently.
- **Memory Management**
  System must allocate and manage memory so multiple jobs can reside in memory at the same time.
- **CPU Scheduling**
  OS must choose among several ready jobs to keep the CPU busy at all times.
- **Device Allocation**
  System must assign devices to jobs and manage device usage without conflicts.

# Time-sharing System

- **Extension of Multiprogramming**

  A time-sharing system is a logical evolution of multiprogramming, enabling interactive use by multiple users.
- **Efficient CPU Usage**

  The CPU rapidly switches between multiple jobs, allowing users to interact with each program as if it were running alone.
- **Multi-user Environment**

  Many users can access and share the system **simultaneously**.
- **Resource Sharing via Scheduling**

  Time-sharing OS uses **CPU scheduling** and **multiprogramming** to allocate small, timed portions of the CPU to each user.
- **Memory Management**

  Jobs are **swapped in and out** of memory to and from the disk as needed.
- **Interactive Communication**

  Real-time interaction between the user and the system:

  When the OS finishes executing a command, it immediately waits for the next input (control statement) from the user's keyboard.
- **Online, On-Demand Access**

  The system must be available **on-demand** for users to access data, applications, and services.

# Desktop Systems

## Personal Computers

- Computer systems designed for **single-user** access and operation.

## Common I/O Devices

- Keyboards, mice, display screens, and small printers that support everyday interaction.

## Convenience & Responsiveness

- Designed for fast performance, ease of use, and immediate feedback to user actions.

## Adopts Technology from Larger OSes

- Desktop systems can incorporate techniques from larger systems.
- Since they are typically used by one person, they require fewer advanced protection and CPU-sharing mechanisms.

## Multiple Operating Systems

- Can run various OS types, including **Windows**, **macOS**, **UNIX**, and **Linux**.

# Multiprocessor System

🔁 **Also Known As:**

- Parallel System
- Tightly Coupled System

💡 **Ideal for:**

- ✓ High-performance computing
- ✓ Real-time applications
- ✓ Fault-tolerant systems

🖥️ **What is a Multiprocessor System?**

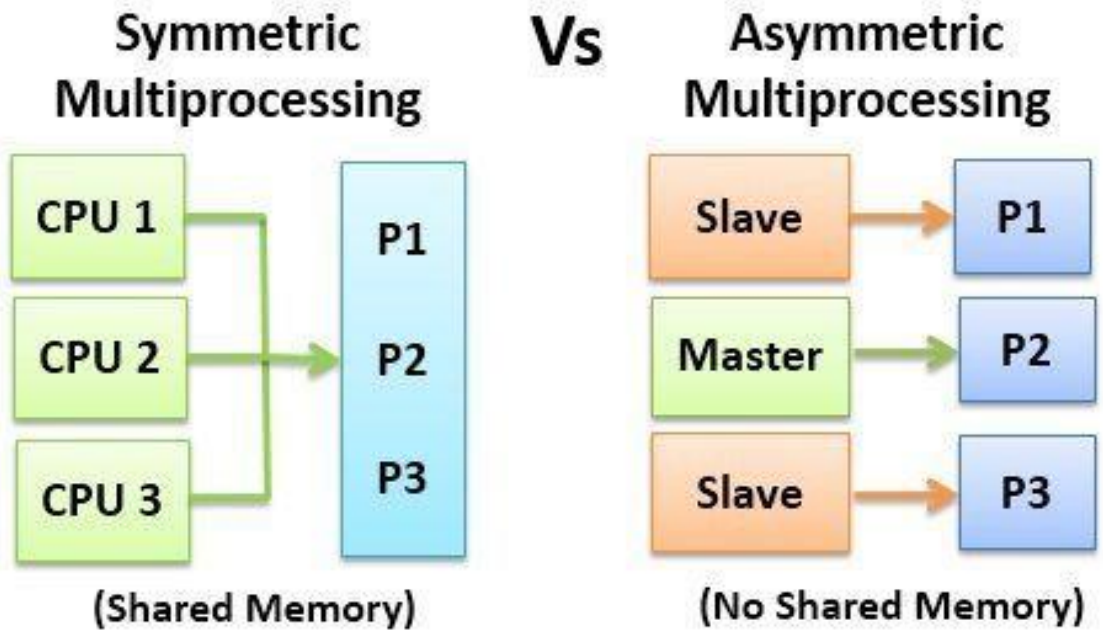
- A computing system with **two or more CPUs** that:
- Work in **close communication**
- Share the **system bus**, **clock**, and sometimes **memory** and **peripherals**
- Operate in a **coordinated** manner to execute tasks concurrently

✅ **Key Advantages:**

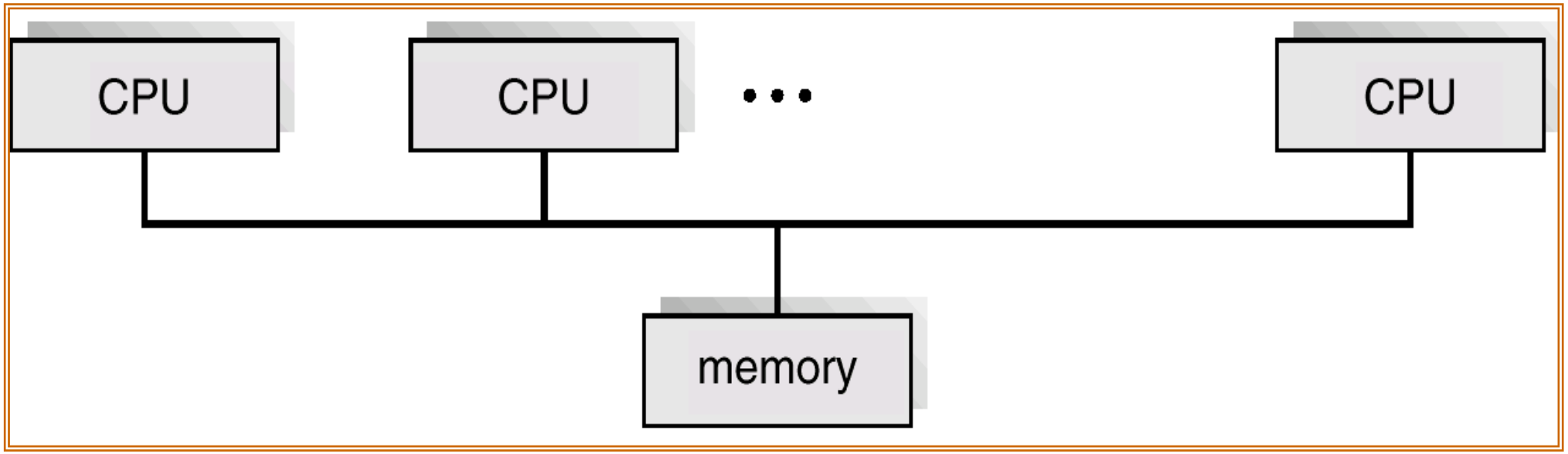
- **Increased Throughput**
  – Multiple CPUs execute tasks simultaneously for better performance
- **Economy of Scale**
  – Shared resources reduce overall system cost
- **Enhanced Reliability**
  – One processor's failure does not bring down the entire system

# Multiprocessor System(cont.)

- Symmetric multiprocessing (SMP)
  - Each processor runs and identical copy of the operating system.
  - Copies communicate with another as needed.
  - Most modern operating systems support SMP.
- Asymmetric multiprocessing
  - Each processor is assigned a specific task;
  - master processor schedules and allocated work to slave processors.
  - More common in extremely large systems
  - Sun's operating system SunOS Ver 4 provides asymmetric multiprocessing.



# Symmetric Multiprocessing Architecture



# Distributed Systems

## 🌐 What is a Distributed System?

- A computing model in which **multiple independent processors** work together and share tasks.

## 🔗 Loosely Coupled System (Distributed Architecture)

- Each processor has its **own local memory**
- Processors communicate through **networks** (high-speed buses, LAN/WAN, etc.)
- This is the **most common form of distributed system**

## Advantages of Distributed Systems

### 🤝 Resource Sharing

- Access shared files, devices, data, and services across multiple machines.

### ⚡ Increased Performance (Load Sharing)

- Workload is distributed among processors, improving computation speed.

### 🛡️ Reliability & Fault Tolerance

- Failure of one node does not bring down the entire system.

### 📫 Enhanced Communication

- Enables efficient data exchange and collaboration across nodes.

# Distributed Systems(cont.)

- Requires networking infrastructure
- Local area networks (*LAN*) or Wide area networks (*WAN*)
- May be either *client-server* or *peer-to-peer* systems

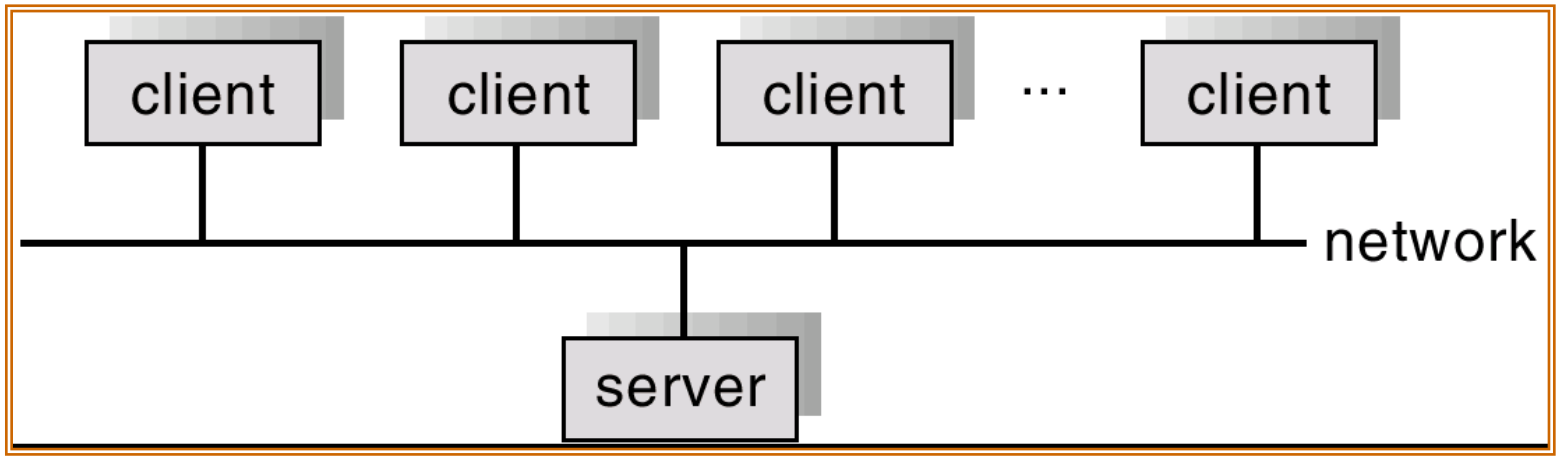



Figure: General Structure of Client-Server

# Clustered Systems

## 🤝 What is Clustering?

- Clustering connects **two or more systems** so they can share storage and work together.

## 🔒 Purpose: High Reliability & Availability

- Clusters improve system uptime by allowing another machine to take over when one fails.

**Types of Clustering :**

### 🟡 Asymmetric Clustering

- One server is **active**, running the application.
- One or more servers remain **standby** and only monitor the active server.
- If the active server fails, a standby node **automatically becomes the new active** server.

### 🟢 Symmetric Clustering

- **All servers (N nodes)** run applications simultaneously.
- More efficient because it uses **all available hardware** instead of keeping servers idle.
- Provides both load sharing and failover capability.

# Real-Time Systems

- A computer system that must respond to inputs **within strict, predefined time limits**.
- Correctness depends on both the **output** and the **time at which the output is produced**.
- Used in applications where **delayed responses are unacceptable**.
- Can be classified as **hard real-time** or **soft real-time** systems.

## ⚙️ Purpose & Use Cases

Real-time systems are used as control devices in dedicated applications such as:

- Scientific experiments
- Medical imaging systems
- Industrial control systems
- Flight control systems
- Display and monitoring systems

## ⏱️ Key Characteristics

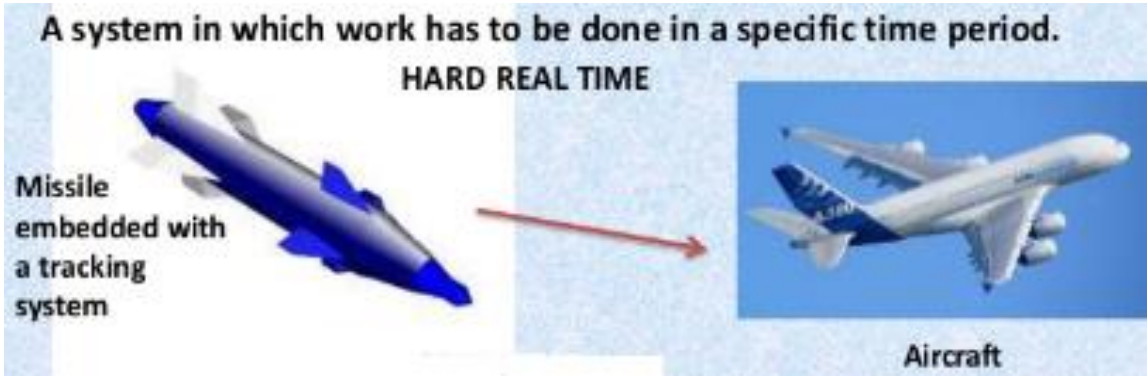
- Operate under **strict, well-defined timing constraints**
- Processing must occur **within set deadlines**, or the system may fail
- Can be categorized into **hard** and **soft** real-time systems

# Real-Time Systems (cont.)

## Types of Real-Time Systems:

### Hard Real-Time Systems

- Guarantees that **critical tasks are completed on time**
- Secondary storage is often **limited or absent**; data kept in short-term memory or ROM
- Not compatible with **time-sharing systems**; general-purpose OSes do not support them
- **Missing deadlines is catastrophic**
- **Examples:**
  - Flight control systems
  - Medical monitoring systems
  - Airbag control systems




### Soft Real-Time Systems

- **Limited utility** in robotics or industrial control
- Can be **integrated with time-sharing systems**
- Useful for applications that require **fast but not strictly guaranteed** response times
- **Missing deadlines is undesirable but not fatal**
- **Examples:**
  - Online databases
  - Multimedia systems
  - Computer games

# Handheld Systems



## Overview

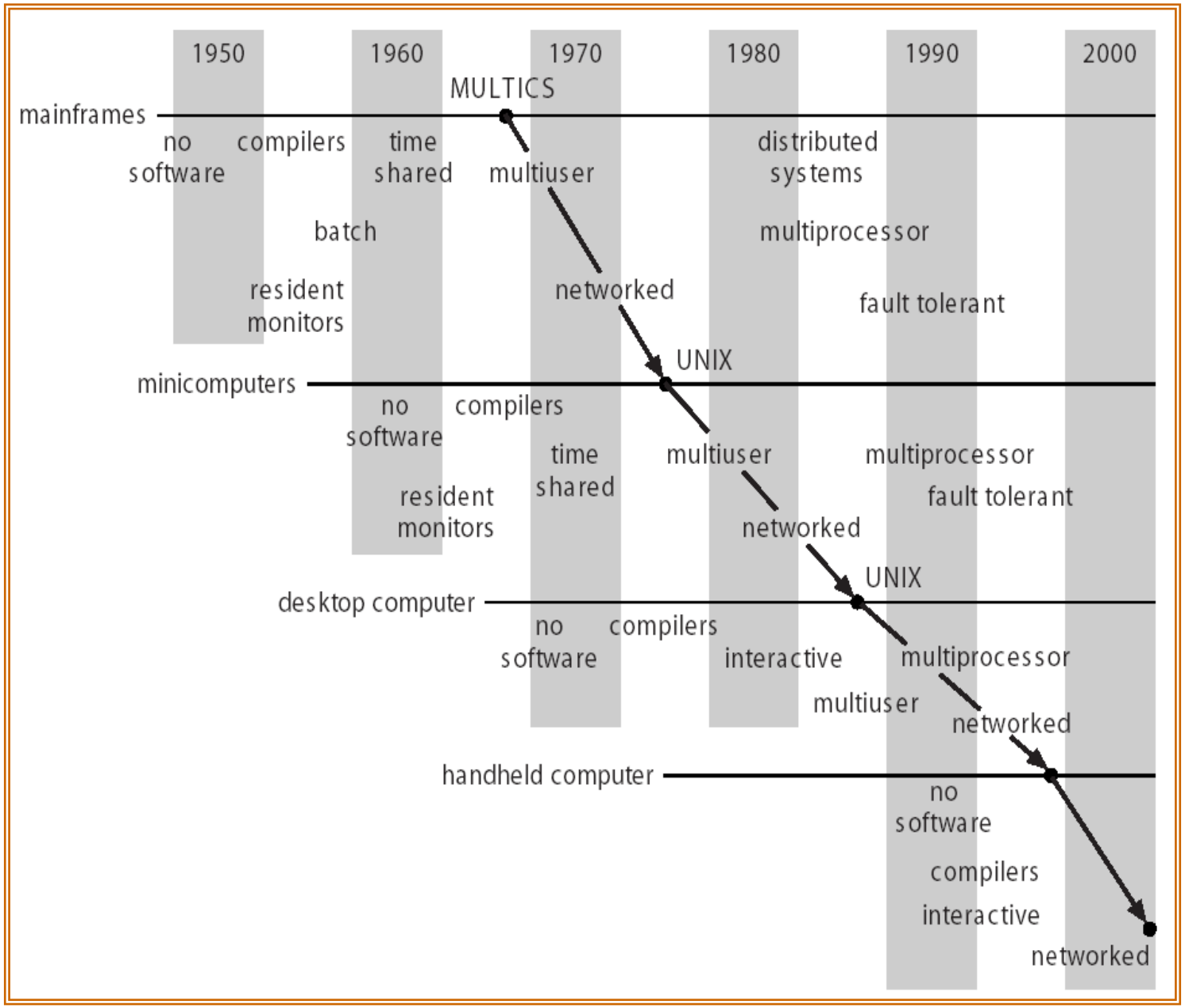
- Also known as **Personal Digital Assistants (PDAs)**
- Most modern handheld devices can **connect to the Internet**
  - Examples: smartphones / cellular phones

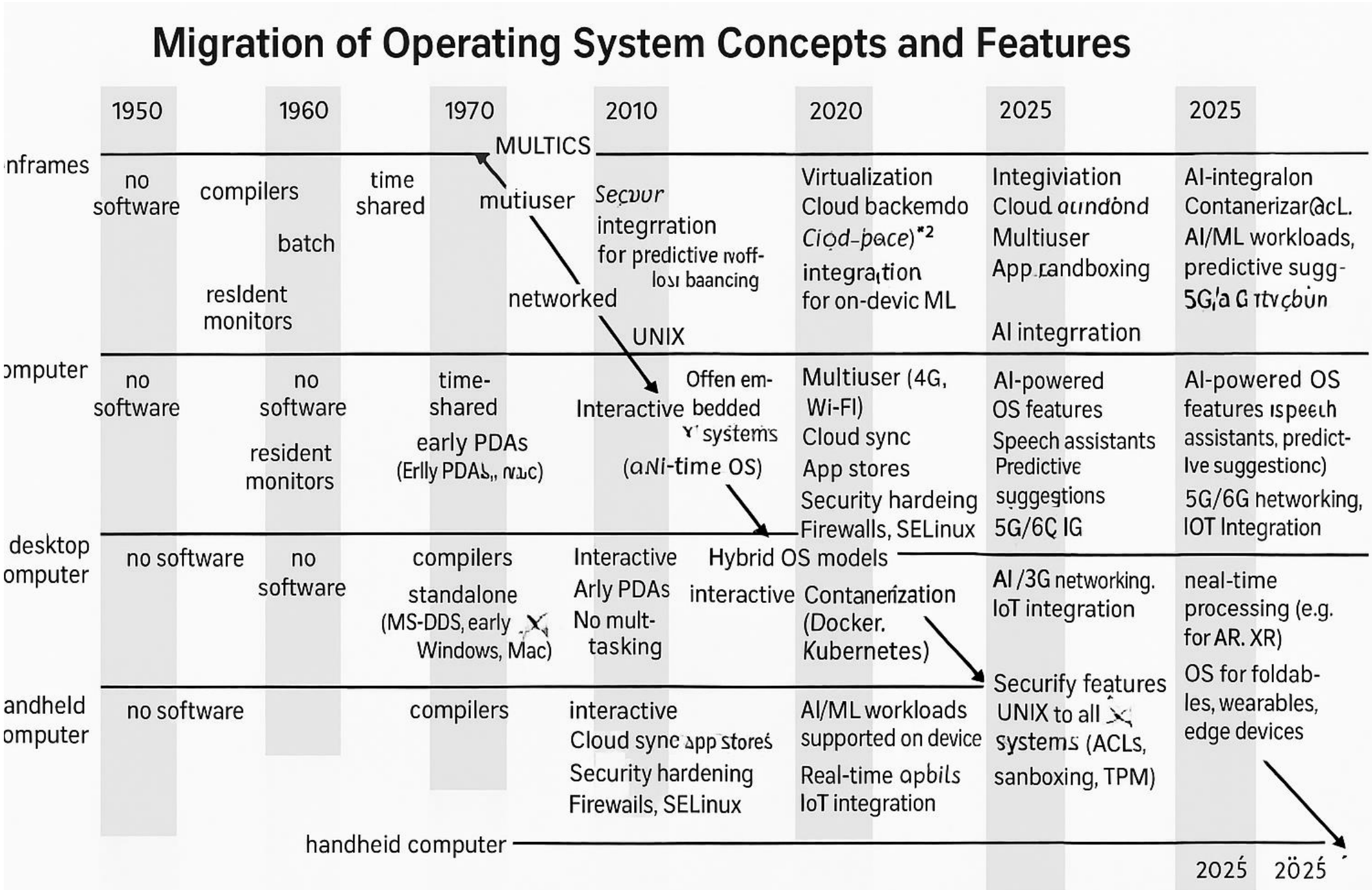


## Issues and Limitations

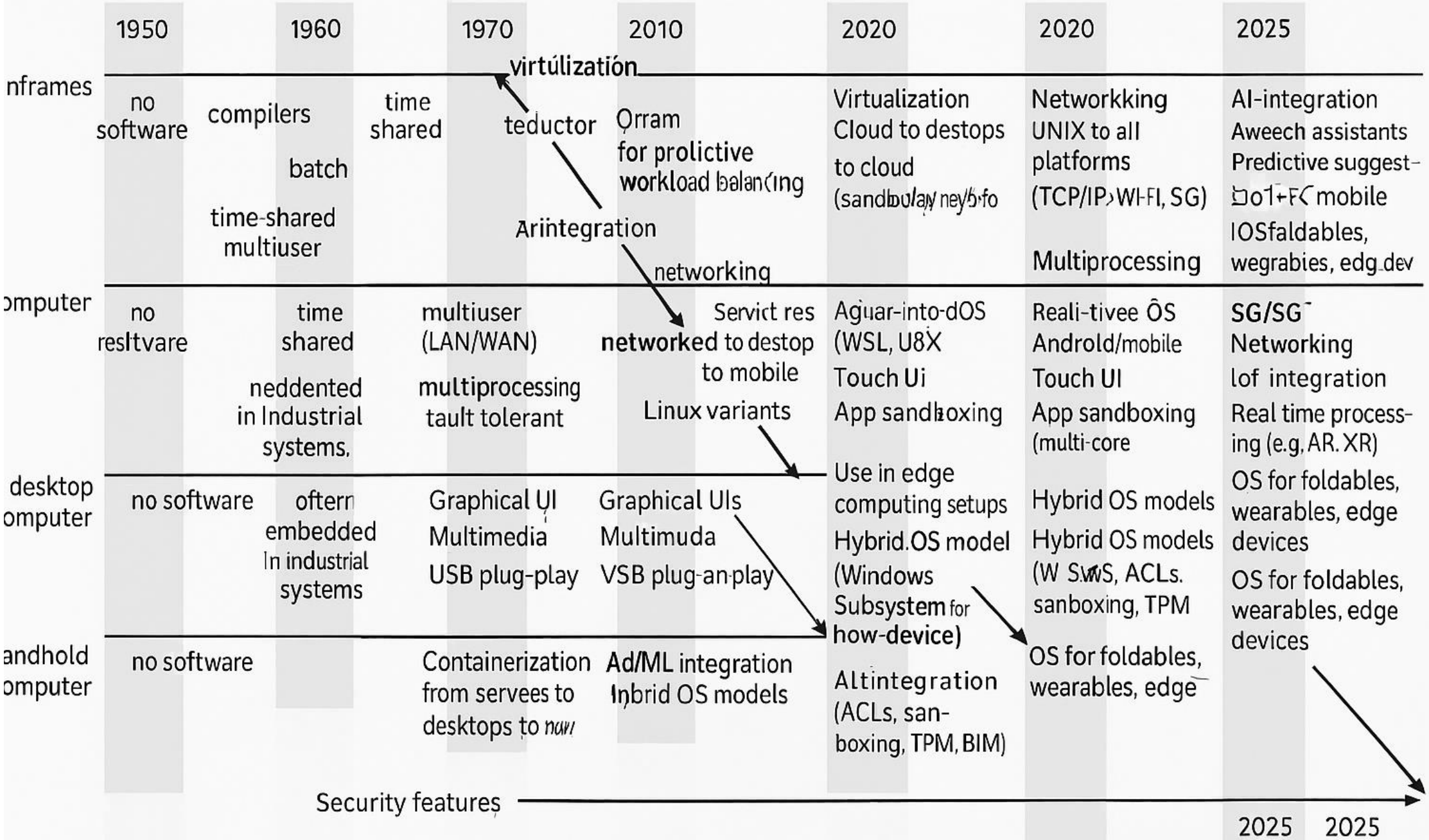
- **Limited memory**
- **Slow processors**
- **Small display screens**

# Migration of Operating System Concepts and Features




Migration of Operating System Concepts and Features
1950
1960
1970
2010
2020
2025
2025
nframes
MULTICS
no software
compilers
batch
resident monitors
time shared
mutiuser
networked
integrration
for predictive
integration
for on-devic ML
Virtualization
Cloud backemdo
Multiuser
App sandboxing
AI integrration
AI-integralon
Containerizar
AI/ML workloads,
predictive sugg-
UNIX
omputer
no software
no software
resident monitors
time-shared
early PDAs
Interactive
Offen em-bedded
systems
(ani-time OS)
Multiuser (4G, Wi-FI)
Cloud sync
App stores
Security hardeing
Firewalls, SELinux
AI-powered OS features
Speech assistants
Predictive suggegtions
5G/6G IG
AI-powered OS features
assistants, predict-
Ive suggestionc)
5G/6G networking,
IOT Integration
desktop omputer
no software
no software
compilers
standalone
(MS-DDS, early Windows, Mac)
Interactive
Arly PDAs
No mult-tasking
Hybrid OS models
interactive
Containerization
(Docker. Kubernetes)
AI /3G networking.
IoT integration
neal-time processing (e.g. for AR. XR)
andheld omputer
no software
compilers
interactive
Cloud sync app stores
Security hardening
Firewails, SELinux
AI/ML workloads
supported on device
Real-time apbils
IoT integration
Securify features
UNIX to all systems (ACLs,
sanboxing, TPM)
OS for foldab-les, wearables, edge devices
handheid computer
2025
2025

# Migration of Operating System Concepts and Features



# 🧠 Migration of Operating System Concepts and Features (1950–2025)

## Mainframes

- **1950s–60s:**
  - No software → Resident monitors → Batch processing
  - Compilers introduced
- **1970s:**
  - **MULTICS**: Time-shared, multiuser, secure OS
- **1980s–90s:**
  - **UNIX**: Networking, multiprocessing, distributed systems
  - Fault tolerance, enhanced scheduling
- **2000–2025:**
  - **Virtualization**: Mainframes now support VMs
  - **Cloud backend** support
  - **AI integration** for predictive workload balancing

# 🧠 Migration of Operating System Concepts and Features (1950–2025)

## Minicomputers

- **1960s–70s:**
  - Resident monitors → Time-shared systems
  - Multiuser support
- **1980s–90s:**
  - UNIX migration
  - Networking and multiprocessor support
- **2000–2025:**
  - Often embedded in **industrial systems**
  - Support for **Linux variants**, real-time OS
  - Used in **edge computing** setups

# 🧠 Migration of Operating System Concepts and Features (1950–2025)

## Desktop Computers

- **1970s–80s:**
  - No software → Compilers → Interactive systems
  - Standalone systems (MS-DOS, early Windows, Mac)
- **1990s:**
  - Multiuser features (Windows NT, Linux)
  - Networking (LAN/WAN)
- **2000s:**
  - **Graphical UIs**, multimedia, USB plug-and-play
  - **Multiprocessing** becomes common (multi-core CPUs)
- **2010s:**
  - **Virtualization** (VMware, VirtualBox)
  - **Cloud integration** (OneDrive, Google Drive)
  - Security hardening, firewalls, SELinux
- **2020s–2025:**
  - **Containerization** (Docker, Kubernetes)
  - **AI/ML workloads** supported on-device
  - **Hybrid OS models**: Windows Subsystem for Linux (WSL), Linux on Mac (Asahi)

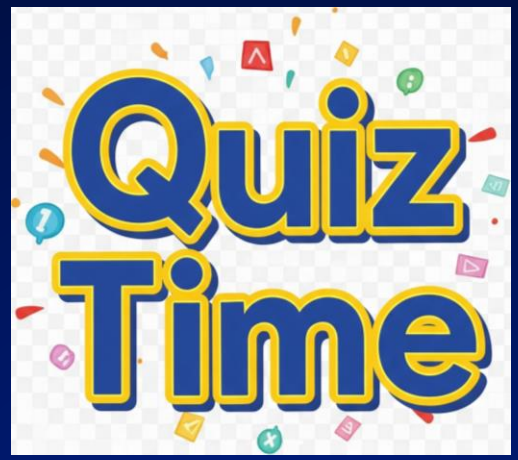
# 🧠 Migration of Operating System Concepts and Features (1950–2025)

## Handheld Devices (Smartphones, Tablets, Wearables)

- **1990s–2000s:**
  - Early PDAs → Basic interactive OS
  - No multitasking
- **2007 onwards:**
  - **iOS, Android**: Touch UI, app sandboxing
  - Compilers and SDKs (Xcode, Android Studio)
- **2010s:**
  - Networked: 4G, Wi-Fi
  - **Cloud sync**, app stores
  - **Security**: biometric auth, encrypted storage
- **2020s–2025:**
  - **AI-powered OS features**: speech assistants, predictive suggestions
  - **5G/6G networking**, IoT integration
  - Real-time processing (e.g., for AR, XR)
  - OS for **foldables**, **wearables**, **edge devices**

# Modern Features Migrated Across All Platforms

| Feature | Where It Migrated |
| --- | --- |
| **Virtualization** | Mainframes → Desktops → Cloud → Mobile (sandboxing) |
| **Networking** | From UNIX → All platforms (TCP/IP, Wi-Fi, 5G) |
| **Multiprocessing** | Servers → Desktops → Mobile (multi-core CPUs) |
| **Real-time OS** | Embedded → Mobile → Edge computing |
| **Cloud Integration** | Desktops/Mobile → Mainframes (backend) |
| **AI Integration** | Desktops → Cloud → Mobile (on-device ML) |
| **Containerization** | Servers → Desktops → Now edge and IoT |
| **Security Features** | UNIX → All systems (ACLs, sandboxing, TPM, biometrics) |




START THE QUIZ

What is the primary purpose of a **mainframe system**?

A) Gaming performance

C) Mobile computing

B) Personal data storage

D) Handling large-scale transaction processing

Next

In a **batch processing system**, when are jobs processed?

A) As soon as they arrive

C) After all jobs are collected into a batch

B) Only during system boot

D) In real-time

Next

What component is mainly responsible for **job scheduling** in a batch system?

A) Control Unit

C) Operating System

B) RAM

D) Input Device

Next

What is the main objective of a multiprogramming system?

A) Reduce memory usage

C) Improve battery performance

B) Provide graphical interface

D) Increase CPU utilization

Next

What technique does a time-sharing system use to handle multiple users?

A) CPU Scheduling and Multiprogramming

C) Batch Processing

B) Priority Queueing

D) Virtual Reality

Next

Which mechanism is crucial for the CPU to efficiently switch between multiple programs in a multiprogrammed system?

A) Saving the state of a running program and loading the state of another to give it CPU control.

C) Connecting to a local area network (LAN)

B) Loading a new application from a CD-ROM

D) Defragmenting the hard drive to improve file access speed

Next

What is a multiprocessor system also known as?

A) Real-time system

C) Tightly coupled system

B) Distributed system

D) Batch system

Well done!

# System Calls

## What Are System Calls?

- Provide the **interface** between user programs and the services offered by the **operating system**.

## How They Are Written

- Usually implemented in **high-level languages** such as **C or C++**.

## How Programs Use Them

- Programs typically access OS services through a high-level **Application Programming Interface (API)** rather than calling system calls directly.

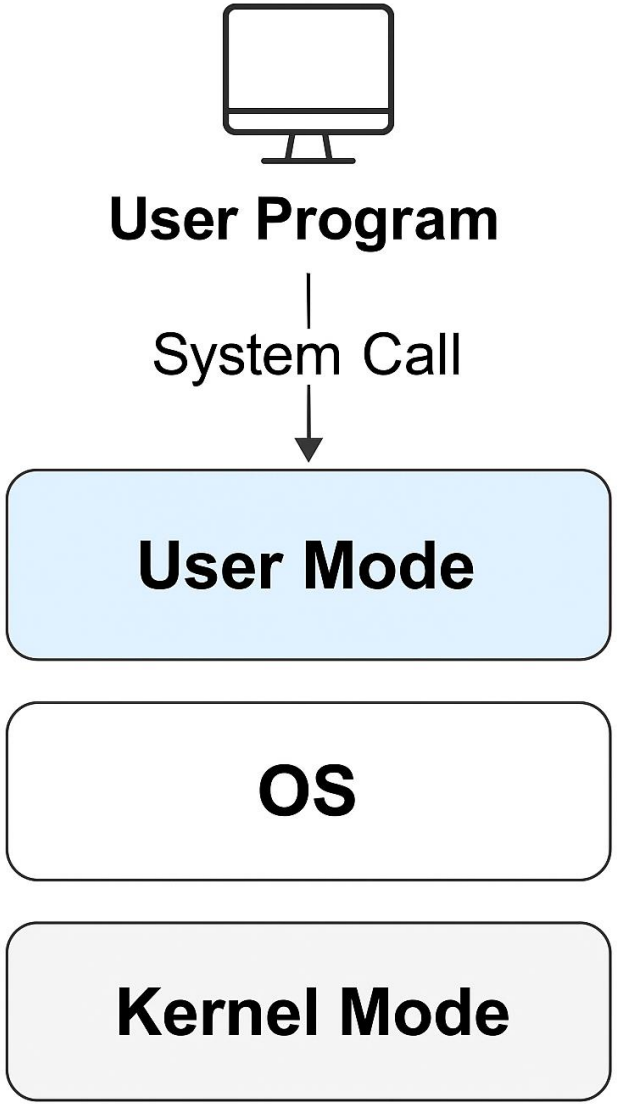
## Common System Call APIs

- **Win32 API** – used by Windows
- **POSIX API** – used by UNIX-like systems (Linux, macOS, etc.)
- **Java API** – used by programs running inside the **Java Virtual Machine (JVM)**

**Simple Example:**

**System Call: `read()`**

- Used by applications to request data from a file or device
- Makes the OS fetch data from storage into memory
- Transfers control to **kernel mode** during execution



# Examples of System Calls

## 1. Process Control

- fork() – create a new process
- exec() – run a new program
- exit() – terminate a process
- wait() – wait for a child process to finish

## 2. File Management

- open() – open a file
- read() – read data from a file
- write() – write data to a file
- close() – close a file

## 3. Device Management

- ioctl() – control device parameters
- read() / write() – interact with device drivers

## 4. Information Maintenance

- getpid() – get current process ID
- alarm() – set a timer
- gettimeofday() – get system time

## 5. Communication

- pipe() – create a communication channel
- send() / recv() – send or receive messages across sockets
- connect() – establish a network connection

**Examples: In Unix/Linux (C)**

```c
int fd = open("file.txt", O_RDONLY);   // system call
read(fd, buffer, 100);                 // system call
close(fd);                             // system call
```

# System Call Implementation

### 🔷 System Call Identification

- Each system call is assigned a **unique number**.
- The **system-call interface** maintains a table indexed by these numbers.

### 🔷 Execution Flow

- The system-call interface:
  - Identifies the requested system call
  - Transfers control to the **OS kernel**
  - Executes the requested service
  - Returns **status and results** to the calling program

### 🔷 Programmer's Perspective

- The caller **does not need to know** how the system call is implemented internally.
- Programmers only need to:
  - Follow the **API**
  - Understand the expected behavior and return values

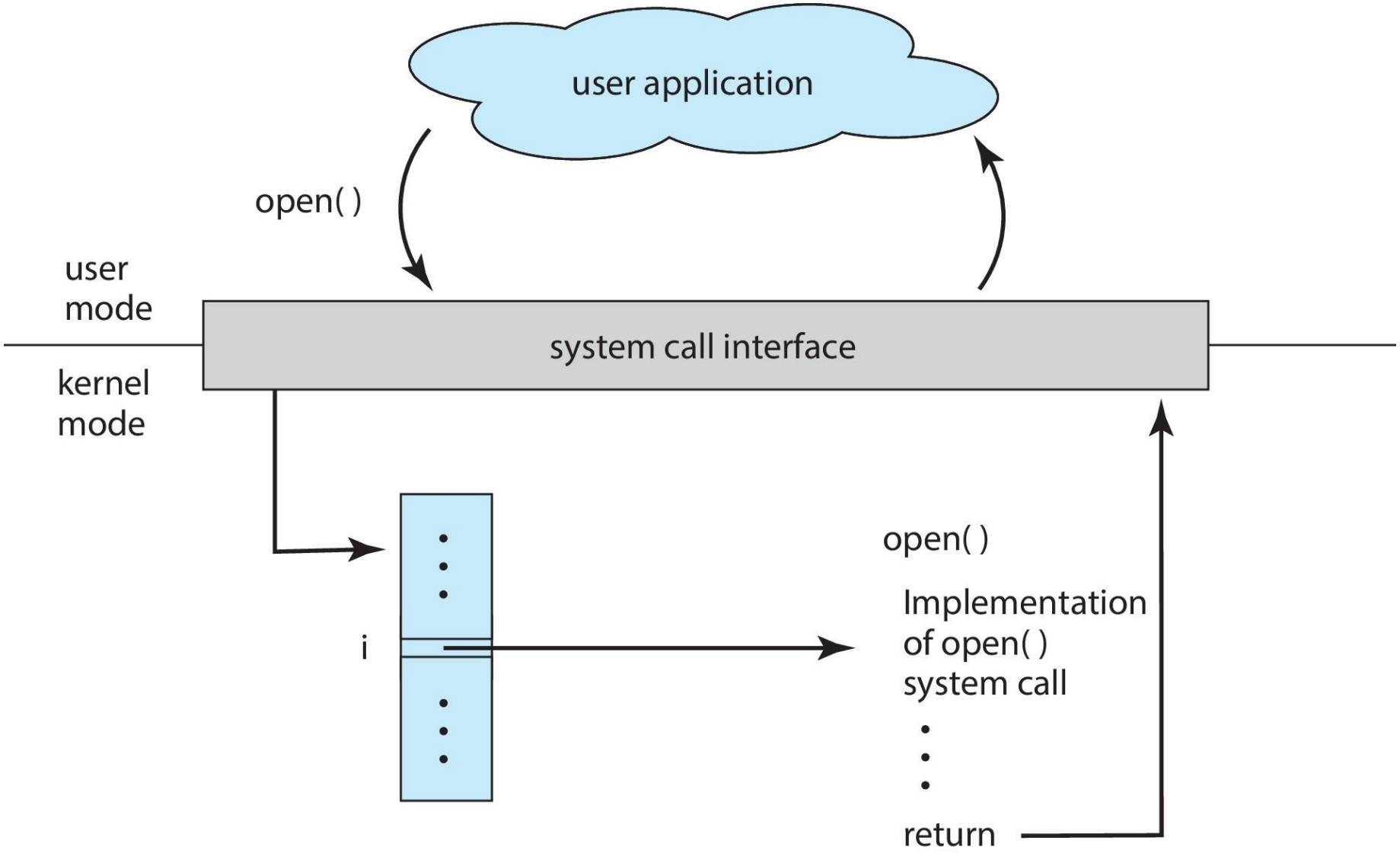
### 🔷 Role of APIs and Libraries

- Most OS-specific details are **hidden by the API**.
- System calls are typically invoked through **runtime support libraries** (standard libraries provided with the compiler).

### ⭐ Key Idea

- **System calls provide a clean and secure interface between user programs and the OS kernel.**

# API – System Call – OS Relationship



# API – System Call – OS Relationship

### 🔷 1. User Application (User Mode)

- A user program requests an OS service using an **API function**
  - Example: open()
- The application runs in **user mode**, where direct hardware access is not allowed.

### 🔷 2. System Call Interface

- The API call triggers a **system call**.
- The **system call interface**:
  - Identifies the requested service using a **system call number**
  - Switches the CPU from **user mode to kernel mode**

### 🔷 3. Kernel Mode Execution

- The OS kernel executes the **actual implementation** of the system call
  - Example: open() system call implementation
- The kernel safely accesses hardware and system resources.

### 🔷 4. Return to User Application

- After execution:
  - The kernel returns the **result or status**
  - Control switches back to **user mode**
  - The application resumes execution

# System Call Parameter Passing

## 🔷 Why Parameter Passing Is Needed

- Identifying a system call is **not sufficient** by itself.
- Most system calls require **additional information (parameters)**.
- The **type and number of parameters** depend on:
  - The operating system
  - The specific system call

### Methods of Passing Parameters to the OS

✅ **1. Register-Based Passing (Simplest)**

- Parameters are passed using **CPU registers**.
- Fast and efficient.
- Limitation: The number of parameters is limited by available registers.

✅ **2. Memory Block / Table Method**

- Parameters are stored in a **block or table in memory**.
- The **address of the block** is passed in a register.
- Used by:
  - **Linux**
  - **Solaris**
- Advantage:
  - Supports **more parameters** than registers alone.

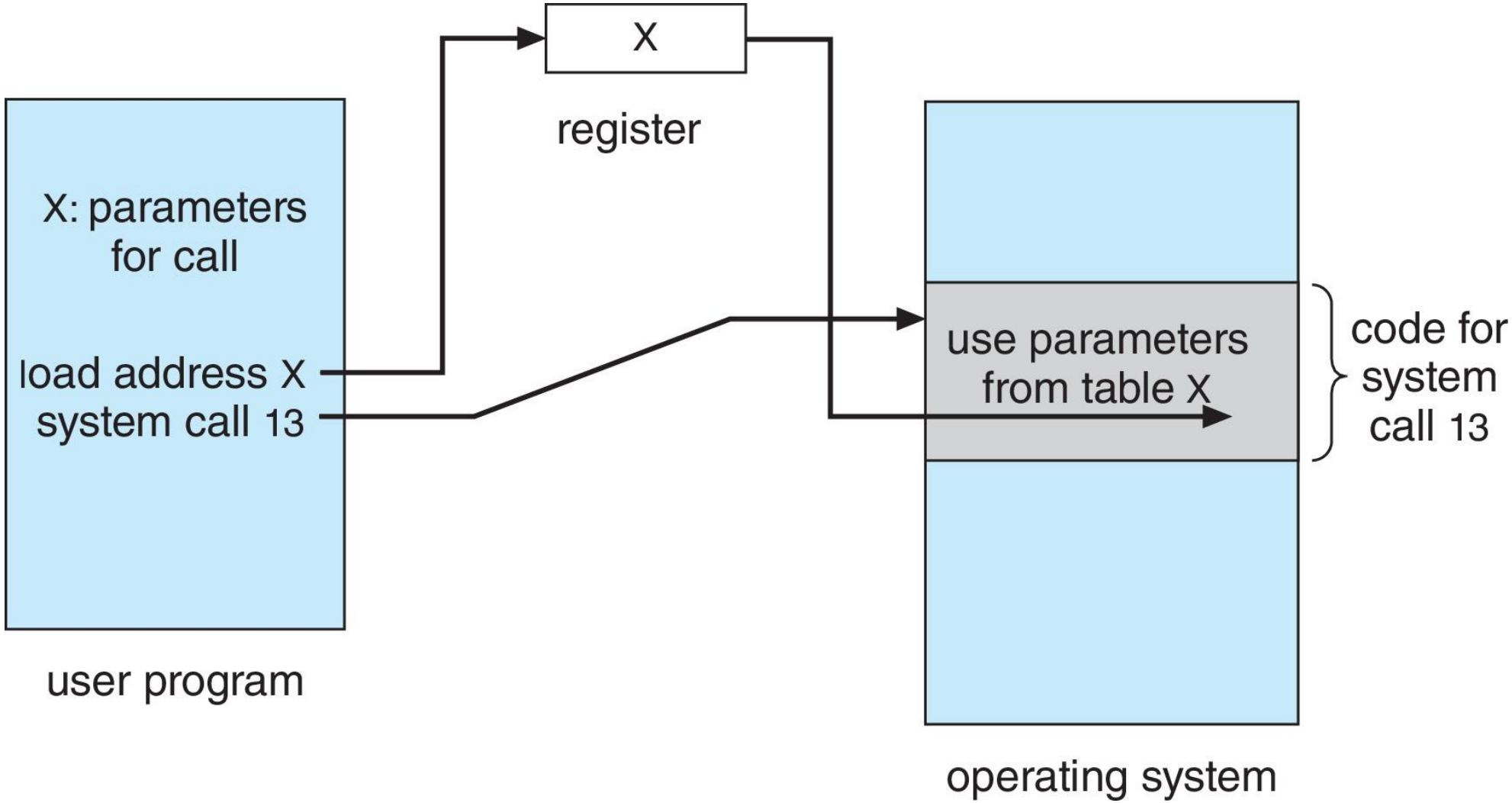
✅ **3. Stack-Based Passing**

- Parameters are **pushed onto the stack** by the user program.
- The OS **pops** the parameters from the stack.
- Advantage: No limit on the **number or size** of parameters.

⭐ **Key Takeaway**

**Registers** → fastest, limited
**Memory block & stack** → flexible, scalable
Modern OSes commonly use **memory block or stack methods**

# Parameter Passing via Table



# Types of System Calls

System calls are broadly classified into **six major categories**:

**1. Process Control**

- Used to manage the execution of processes.
- Create, terminate, and control processes
- Load and execute programs
- **Examples:** fork(), exec(), exit(), wait()

**2. File Management**

- Used to handle files and directories.
- Create, open, read, write, and delete files
- Manage file attributes
- **Examples:** open(), read(), write(), close(), unlink()

**3. Device Management**

- Used to control I/O devices.
- Request and release devices
- Read/write device data
- **Examples:** ioctl(), read(), write()

# Types of System Calls (Cont.)

**4. Information Maintenance**

- Used to get or set system and process information.
- Retrieve time, date, system data
- Modify process attributes
- **Examples:** getpid(), alarm(), time(), uname()

**5. Communication**

- Used for inter-process communication (IPC).
- Enable data exchange between processes
- Used in networking and shared memory
- **Examples:** pipe(), socket(), send(), recv()

**6. Protection**

- Used to control access and permissions.
- Set user IDs and file permissions
- Enforce security and protection
- **Examples:** chmod(), chown(), setuid()

**One-Line Summary**

System calls provide controlled access to OS services related to processes, files, devices, communication, and security.

# Types of System Calls (Cont.)

## 1. Process Control

- Process control system calls are used to **create, manage, and terminate processes**.
- **Create & terminate processes**
  - Create a new process
  - End or abort a running process
- **Program execution**
  - Load a program into memory
  - Execute a program
- **Process attributes**
  - Get process attributes (PID, priority, state)
  - Set process attributes (priority, permissions)
- **Process synchronization**
  - Wait for a specific time
  - Wait for an event
  - Signal an event to another process
- **Memory management**
  - Allocate memory
  - Free allocated memory
  - Dump memory in case of errors
- **Debugging support**
  - Support debugging
  - Enable single-step execution
  - Help identify bugs
- **Concurrency control**
  - Locks to manage access to shared data
  - Prevent race conditions between processes

 **Examples :** fork(), exec(), exit(), wait(), kill(), getpid()

## One-Line Summary

- **Process control system calls manage the lifecycle, execution, synchronization, and resources of processes.**

# Types of System Calls (Cont.)

## 2. File Management

- File management system calls handle **file creation, access, and modification**.
- Create and delete files
- Open and close files
- Read, write, and reposition file pointers
- Get and set file attributes (permissions, size, ownership)
- **Examples:**
  open(), close(), read(), write(), lseek(), unlink()

## 3. Device Management

- Device management system calls control **input/output devices**.
- Request and release devices
- Read and write data to devices
- Reposition device pointers
- Get and set device attributes
- Logically attach or detach devices
- **Examples:**
  read(), write(), ioctl()

## ⭐ Key Difference (Quick Recall)

- **File management** → works on files stored on disk
- **Device management** → works on hardware devices (printers, disks, keyboards)

## One-Line Summary

- **File and device management system calls provide controlled access to files and hardware devices.**

# Types of System Calls (Cont.)

## 4. Information Maintenance

- These system calls are used to **retrieve or modify system-related information**.
- Get or set system **time and date**
- Get or set **system data**
- Get and set **process, file, or device attributes**
  (e.g., permissions, ownership, configuration)
- **Examples:** time(), gettimeofday(), uname(), stat(), chmod()

## 5. Communications

- Communication system calls support **data exchange between processes or systems**.
- Create and delete communication connections
- Send and receive messages using:
  - **Message-passing model**
    - Communication by **host name** or **process name**
    - Typically follows **client–server model**
  - **Shared-memory model**
    - Create and access shared memory regions
- Transfer communication **status information**
- Attach and detach **remote devices**
- **Examples:** pipe(), socket(), send(), recv(), shmget(), shmat()

## ⭐ Quick Comparison

- **Information Maintenance** → manages system & attribute data
- **Communications** → enables inter-process and network communication

## One-Line Summary

- **These system calls allow processes to exchange data and manage system information efficiently.**

# Types of System Calls (Cont.)

## 6. Protection

- Protection system calls ensure **security and controlled access** to system resources.
- Control access to **files, devices, and memory**
- Get and set **permissions** for users and groups
- Allow or deny **user access** to system resources
- Enforce **authentication and authorization policies**
- **Examples (Linux):**
  chmod() – change file permissions
  chown() – change file ownership
  umask() – set default permission mask
  access() – check user access rights

## ⭐ Why Protection System Calls Are Important

- Prevent **unauthorized access**
- Protect system integrity and user data
- Support **multi-user operating systems**

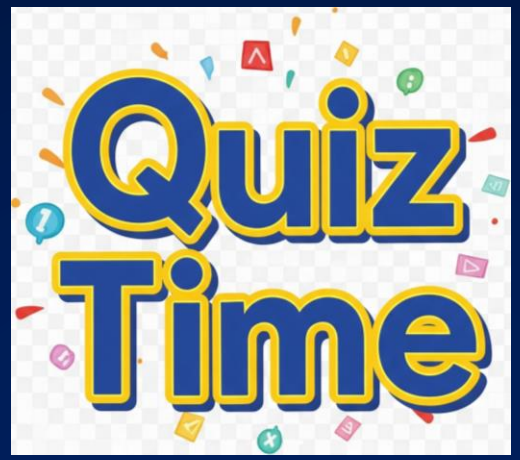
## One-Line Summary

- **Protection system calls provide security by regulating access to system resources and user privileges.**

# Examples of Windows and Unix System Calls

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

| | Windows | Unix |
|---|---|---|
| **Process control** | `CreateProcess()`<br>`ExitProcess()`<br>`WaitForSingleObject()` | `fork()`<br>`exit()`<br>`wait()` |
| **File management** | `CreateFile()`<br>`ReadFile()`<br>`WriteFile()`<br>`CloseHandle()` | `open()`<br>`read()`<br>`write()`<br>`close()` |
| **Device management** | `SetConsoleMode()`<br>`ReadConsole()`<br>`WriteConsole()` | `ioctl()`<br>`read()`<br>`write()` |
| **Information maintenance** | `GetCurrentProcessID()`<br>`SetTimer()`<br>`Sleep()` | `getpid()`<br>`alarm()`<br>`sleep()` |
| **Communications** | `CreatePipe()`<br>`CreateFileMapping()`<br>`MapViewOfFile()` | `pipe()`<br>`shm_open()`<br>`mmap()` |
| **Protection** | `SetFileSecurity()`<br>`InitlializeSecurityDescriptor()`<br>`SetSecurityDescriptorGroup()` | `chmod()`<br>`umask()`<br>`chown()` |




START THE QUIZ

# What is a system call?

A) A user-level program

C) A hardware interrupt

B) A compiler instruction

D) An interface between user programs and the OS kernel

Next

Which mode does a system call execute in?

A) User mode

C) Kernel mode

B) Supervisor mode only

D) Virtual mode

Next

Which system call is used to create a new process in UNIX/Linux?

A) wait()

C) fork()

B) exec()

D) exit()

Next

Which parameter-passing method stores parameters in a memory block and passes its address in a register?

A) Stack method

C) Register method

B) Cache method

D) Table (block) method

Next

Which of the following best describes the purpose of system calls?

A) To provide controlled access to OS services

C) To manage hardware directly by users

B) To compile user programs

D) To replace application programs

Next

Which of the following is a protection-related system call?

A) chmod()

C) read()

B) fork()

D) socket()

Next

After executing fork() system call, how many processes exist?

A) 0

C) 2

B) 1

D) Depends on OS

Well done!


Thank
you!